

# **Benchmarking Lightweight CNN for Real-Time Classification of Bangladeshi Mango Varieties on Edge Devices**

A Project Report submitted to the  
Department of Computer Science and Engineering, Jahangirnagar University  
in partial fulfillment of the requirements for the degree of  
B.Sc. in Computer Science and Engineering

By

Abdullah Nazmus-Sakib, Redwana Jahan

Exam Roll: 210903, 210879

Registration No: 20210652888, 20210652864

Session: 2020 - 2021

Supervised by

Professor Dr. Liton Jude Rozario

Department of Computer Science and Engineering



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING  
JAHANGIRNAGAR UNIVERSITY  
DECEMBER 2026

# Abstract

Mango (*Mangifera indica* L.) is one of most valuable fruit crops in Bangladesh, making accurate variety identification essential for quality control, pricing, and post-harvest management. Traditional identification depends on visual inspection by experts, which is often time-consuming and prone to errors, particularly for morphologically similar varieties. Although deep learning has achieved remarkable success in image classification, many high-performing convolutional neural networks (CNNs) are too computationally demanding for practical deployment on mobile and edge devices. This study proposes a lightweight **Edge AI** framework for real-time classification of indigenous Bangladeshi mango varieties. Five transfer learning-based CNN models—**MobileNetV2**, **MobileNetV3-Small**, **EfficientNetV2B0**, **DenseNet121**, and **NASNetMobile**—were trained and evaluated on the publicly available **Mangifera2012** dataset using a unified training and evaluation pipeline. For practical implementation, the selected MobileNetV3-Small model was converted to **TensorFlow Lite (TFLite)** using **Float16 quantization** and integrated into a **Flutter-based Android application** capable of fully offline inference. The application enables users to classify mango varieties directly on a smartphone and instantly displays the predicted class with its confidence score, eliminating the need for internet connectivity. Overall, this research presents an end-to-end Edge AI pipeline, from model benchmarking to mobile deployment, for automated mango variety classification. The proposed framework demonstrates the feasibility of deploying lightweight deep learning models for real-time precision agriculture in resource-limited environments.

**Keywords:** Edge Artificial Intelligence (Edge AI), Deep Learning, Transfer Learning, MobileNetV3-Small, Convolutional Neural Network (CNN), TensorFlow Lite (TFLite), Flutter, Precision Agriculture, Mango Variety Classification, Computer Vision, Mobile Computing.

# Declaration

The The research work entitled "**Benchmarking Lightweight CNN for Real-Time Classification of Bangladeshi Mango Varieties on Edge Devices**" has been carried out in the Department of Computer Science and Engineering, Jahangirnagar University, under the supervision of **Professor Dr. Liton Jude Rozario**. This work is original and has been completed in accordance with the regulations of Jahangirnagar University.

We declare that this thesis is our own work and has not been submitted, either in whole or in part, to any other institution for the award of any degree, diploma, or other academic qualification. All sources of information, ideas, and materials used in this research have been properly acknowledged through appropriate citations and references.

We understand the University's policy regarding academic integrity and plagiarism and affirm that this thesis does not contain any plagiarized material except where proper acknowledgment has been provided.

## **Submitted By**

**Abdullah Nazmus-Sakib**

Exam Roll: 210903

**Redwana Jahan**

Exam Roll: 210879

---

(Candidate)

## **Counter Signed by**

**Professor Dr. Liton Jude Rozario**

Department of Computer Science and Engineering  
Jahangirnagar University

---

(Supervisor)

# Acknowledgement

First and foremost, we express our sincere gratitude to **Almighty**, whose endless blessings, guidance, and mercy enabled us to complete this research successfully.

We would like to convey our deepest appreciation to our respected supervisor, **Professor Dr. Liton Jude Rozario, Department of Computer Science and Engineering, Jahangirnagar University**, for his invaluable guidance, continuous encouragement, insightful suggestions, and constructive feedback throughout every stage of this research. His support and expertise greatly contributed to the successful completion of this work.

We are also grateful to all the faculty members and staff of the **Department of Computer Science and Engineering, Jahangirnagar University**, for providing an excellent academic environment and the resources necessary to carry out this research.

We sincerely acknowledge the developers and researchers who made the **Mangifera2012** dataset publicly available. Their contribution provided a valuable foundation for this study. We also extend our appreciation to the open-source communities behind **TensorFlow**, **TensorFlow Lite**, **Flutter**, and other software libraries that supported the implementation and evaluation of our proposed system.

Finally, we express our heartfelt gratitude to our parents, family members, and friends for their constant encouragement, patience, and unwavering support throughout our academic journey. Their motivation inspired us to overcome challenges and successfully complete this research.

We hope that this work will contribute to the advancement of **Edge Artificial Intelligence**, **computer vision**, and **precision agriculture**, and serve as a useful reference for future research in intelligent agricultural applications.

# CONTENTS

|                                             |             |
|---------------------------------------------|-------------|
| <b>Abstract</b>                             | <b>ii</b>   |
| <b>Declaration</b>                          | <b>iii</b>  |
| <b>Acknowledgement</b>                      | <b>iv</b>   |
| <b>List of Figures</b>                      | <b>viii</b> |
| <b>List of Tables</b>                       | <b>ix</b>   |
| <b>List of Algorithms</b>                   | <b>x</b>    |
| <br>                                        |             |
| <b>1 Introduction</b>                       | <b>1</b>    |
| 1.1 Overview                                | 1           |
| 1.2 Problem Statement                       | 2           |
| 1.3 Research Objectives                     | 4           |
| <b>2 Literature Review</b>                  | <b>6</b>    |
| 2.1 Global Research on Mango Classification | 6           |
| 2.2 Lightweight CNNs in Agriculture         | 8           |
| 2.3 Gaps in Existing Studies                | 11          |
| 2.4 Summary of Research Gap                 | 14          |
| <b>3 Background Knowledge</b>               | <b>16</b>   |
| 3.1 Machine Learning                        | 16          |
| 3.2 Deep Learning                           | 17          |
| 3.3 Convolutional Neural Network (CNN)      | 19          |
| 3.4 Edge Artificial Intelligence (Edge AI)  | 23          |
| <b>4 Methodology</b>                        | <b>26</b>   |
| 4.1 Model Architecture Selection            | 26          |
| 4.2 Mathematical Representation             | 27          |
| 4.3 MobileNetV2                             | 30          |
| 4.4 MobileNetV3-Small                       | 31          |

|                                                                              |           |
|------------------------------------------------------------------------------|-----------|
| 4.5 EfficientNetV2B0 .....                                                   | 34        |
| 4.6 NASNetMobile .....                                                       | 36        |
| 4.7 DenseNet121 .....                                                        | 38        |
| 4.8 Dynamic Preprocessing Implementation .....                               | 41        |
| 4.8 Dynamic Preprocessing Workflow.....                                      | 42        |
| 4.10 Integration with Data Augmentation .....                                | 43        |
| 4.11 Architectural Input Disparities .....                                   | 43        |
| 4.12 Pipeline Design .....                                                   | 44        |
| 4.13 Training Pipeline .....                                                 | 45        |
| 4.14 Pipeline Automation .....                                               | 46        |
| 4.15 Methodology Flowchart .....                                             | 46        |
| 4.15 Operational Workflow .....                                              | 49        |
| <b>5 Tools and Datasets .....</b>                                            | <b>57</b> |
| 5.1 Dataset Description .....                                                | 51        |
| 5.2 Dataset Composition .....                                                | 52        |
| 5.3 Data Preprocessing and Augmentation .....                                | 53        |
| 5.4 Architecture-Specific Input Preprocessing.....                           | 54        |
| 5.5 Data Augmentation.....                                                   | 54        |
| 5.6 Training and Validation Data Generation .....                            | 55        |
| 5.6 Workflow of the Preprocessing Pipeline .....                             | 55        |
| <b>6 Mobile System Implementation .....</b>                                  | <b>56</b> |
| 6.1 Cross-Platform Flutter Application Architecture .....                    | 56        |
| 6.2 Mathematical Representation .....                                        | 58        |
| 6.3 ByteStream Conversion .....                                              | 58        |
| 6.4 Prediction Result Visualization .....                                    | 59        |
| <b>7 Result and Discussion .....</b>                                         | <b>71</b> |
| 7.1 Performance Benchmarking .....                                           | 71        |
| 7.2 Comparative Analysis .....                                               | 73        |
| 7.3 Deployment-Oriented Model Selection.....                                 | 74        |
| 7.4 Analysis Confusion Matrix and Classification report for each model ..... | 75        |

|                                           |               |
|-------------------------------------------|---------------|
| <b>8 Conclusion And Future Work .....</b> | <b>76</b>     |
| 8.1 Research Summary .....                | 76            |
| 8.2 Future Work.....                      | 77            |
| <br><b>References .....</b>               | <br><b>78</b> |

# LIST OF FIGURES

|                                                                                             |    |
|---------------------------------------------------------------------------------------------|----|
| 4.1: The architecture of the classifier .....                                               | 27 |
| 4.2: Architecture of a MobileNetV2 residual block .....                                     | 29 |
| 4.4: Architecture of a MobileNetV3-Small residual block .....                               | 31 |
| 4.5: Architecture of a EfficientNetV2B0 residual block .....                                | 34 |
| 4.6: Architecture of a NasNetMobile residual block .....                                    | 37 |
| 4.7: Architecture of a DenseNet121 residual block .....                                     | 39 |
| 4.9: Architecture of Dynamic Preprocessing Workflow.....                                    | 42 |
| 4.10: Architecture of Pipeline Design .....                                                 | 44 |
| 4.11: Architecture of a Automated Pipeline Design .....                                     | 46 |
| 4.12: Architecture of Methodology Flowchart .....                                           | 47 |
| 4.13: Architecture of Operational Workflow .....                                            | 49 |
| 5.6: Architecture of Preprocessing Pipeline .....                                           | 55 |
| 6.3: Comparative Analysis of Class-wise F1-Scores Across Lightweight CNN Architecture ..... | 59 |
| 6.4.1: Confusion Matrix of MobileNetV2 .....                                                | 60 |
| 6.4.2: Classification Report Heatmap for MobileNetV2 .....                                  | 61 |
| 6.4.3: Confusion Matrix of MobileNetV3-Small.....                                           | 62 |
| 6.4.4: Classification Report Heatmap for MobileNetV3-Small .....                            | 63 |
| 6.4.5: Confusion Matrix of EfficientNetV2-B0.....                                           | 64 |
| 6.4.6: Classification Report Heatmap for EfficientNetV2-B0.....                             | 65 |
| 6.4.7: Confusion Matrix of NASNetMobile .....                                               | 66 |
| 6.4.8: Classification Report Heatmap for NASNetMobile .....                                 | 67 |
| 6.4.9: Confusion Matrix of DenseNet121 .....                                                | 68 |
| 6.4.10: Classification Report Heatmap for DenseNet121 .....                                 | 69 |
| 7.1: Cross-Platform Flutter Application Architecture .....                                  | 71 |
| 7.2: TensorFlow Lite Interpreter Flow.....                                                  | 72 |
| 7.3: Image Processing Pipeline .....                                                        | 72 |
| 7.5: Image to ByteBuffer Pipeline .....                                                     | 73 |
| 7.8: Image Processing Algorithm .....                                                       | 74 |



# List of Tables

|                                                                        |    |
|------------------------------------------------------------------------|----|
| 2.1: Comparison of Popular Lightweight CNN Architectures .....         | 11 |
| 2.2: Research Gaps Identified from Existing Literature .....           | 13 |
| 2.3: Summary of Identified Research Gaps and Contributions .....       | 15 |
| 4.1 Selection Criteria for CNN Benchmarking .....                      | 28 |
| 4.2 Architecture-Specific Preprocessing Functions .....                | 28 |
| 4.3 Architectural Input Preprocessing Requirements .....               | 43 |
| 4.4 Operational Stages of the Edge AI Inference Pipeline .....         | 50 |
| 5.1 Distribution of Mango Varieties in the Mangifera2012 Dataset ..... | 51 |
| 5.2 Class-wise Distribution of the Mangifera2012 Dataset .....         | 52 |
| 6.1 Comparative Performance of Evaluated CNN Architectures .....       | 52 |

# LIST OF ALGORITHMS

|                                                           |    |
|-----------------------------------------------------------|----|
| 3.1: MobileNetV2 Algorithm .....                          | 30 |
| 3.2: MobileNetV3-Small Algorithm .....                    | 31 |
| 3.3: EfficientNetV2B0 Algorithm .....                     | 34 |
| 3.4: DenseNet121 Algorithm .....                          | 38 |
| 3.5: NASNetMobile Algorithm .....                         | 36 |
| 3.6: Transfer Learning Algorithm.....                     | 26 |
| 3.7: Dynamic Image Preprocessing Algorithm .....          | 41 |
| 3.8: CNN Training Pipeline Algorithm .....                | 45 |
| 3.9: TensorFlow Lite Float16 Quantization Algorithm ..... | 71 |
| 3.10: Offline Mango Variety Prediction Algorithm .....    | 73 |

# CHAPTER 1

## INTRODUCTION

### 1.1 OVERVIEW

Agriculture remains one of the most important pillars of the economy of Bangladesh. Among the country's diverse agricultural products, mango (*Mangifera indica* L.) occupies a unique position due to its high commercial value, nutritional benefits and widespread cultivation across northwestern regions, particularly Rajshahi and Chapainawabganj. These regions are renowned for producing numerous indigenous mango varieties, including Amrapali, Langra, Fazlee, Harivanga, Mollika, Katimon, Bari-4, Bari-7, Kanchon Langra, and Nilambori, each possessing distinct physical characteristics, taste profiles, market demand, and economic value [1].

Farmers, traders, wholesalers and market inspectors typically identify mango varieties through visual observation based on characteristics such as fruit shape, skin color, size, texture, and maturity stage. While experienced workers often achieve acceptable classification accuracy, this process remains inherently subjective and inconsistent. Environmental conditions, worker fatigue, varying lighting conditions, and the high visual similarity among several indigenous cultivars frequently result in misclassification, affecting pricing, quality assurance, inventory management, and consumer confidence [2]. These limitations become even most pronounced during peak harvesting seasons, where thousands of mangoes must be sorted within a limited timeframe.

As Bangladesh continues to modernize its agricultural practices, there is an increasing demand for intelligent technologies capable of automating these labor-intensive operations while maintaining consistent classification performance. Convolutional Neural Networks (CNNs) have become the dominant approach for visual recognition tasks because of their ability to automatically learn hierarchical feature representations directly from raw images without requiring handcrafted feature engineering. Images captured in the field must be transmitted to remote servers for processing before predictions are returned to the user. Edge Artificial Intelligence (Edge AI) has recently emerged as an effective solution to this.

networks—MobileNetV2, MobileNetV3-Small, EfficientNetV2B0, DenseNet121, and NASNetMobile—are trained and evaluated under an identical experimental framework using the publicly available Mangifera2012 dataset. The benchmarking process compares each architecture based on classification accuracy, precision, recall, F1-score, model storage requirements, and inference latency to determine the most appropriate architecture for real-world edge deployment.

Following the comparative evaluation, the selected model undergoes additional deployment-oriented optimization through TensorFlow Lite (TFLite) conversion using post-training Float16 quantization. This optimization reduces storage requirements while preserving classification performance, making the model suitable for execution on modern Android smartphones without requiring specialized AI hardware. The optimized model is subsequently integrated into a fully functional Flutter-based mobile application named **AmmChini AI**, which enables users to capture mango images using the device camera or select existing photographs from the gallery and perform entirely offline variety classification directly on the mobile device.

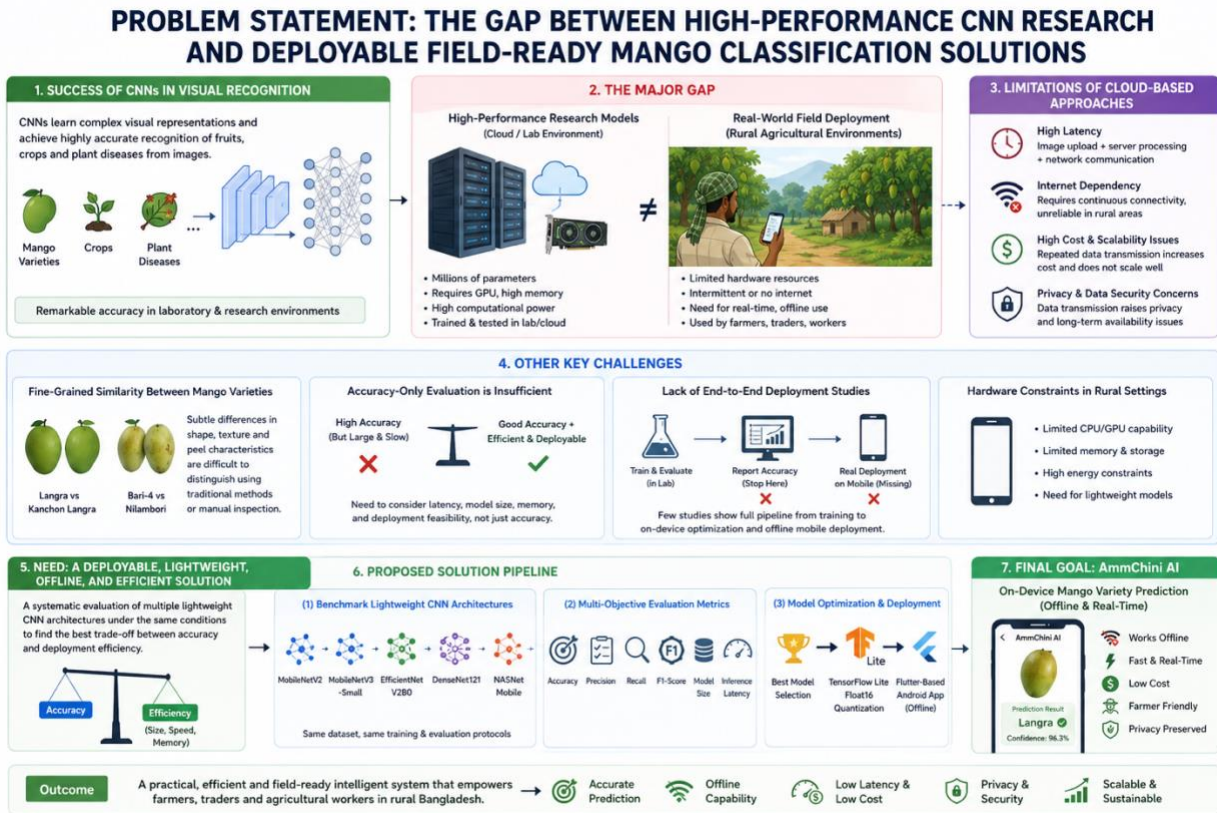
## 1.2 PROBLEM STATEMENT

Convolutional Neural Networks (CNNs) have demonstrated remarkable success in learning complex visual representations, enabling highly accurate recognition of fruits, crops, and plant diseases directly from image data [1]. Nevertheless, despite these technological advances, most existing mango classification systems remain confined to research laboratories and cloud based infrastructures, limiting their practical applicability in real-world agricultural environments. The gap between high-performance deep learning research and deployable field-ready solutions remains a major challenge. State-of-the-art CNN models, including ResNet, DenseNet, EfficientNet, and Vision Transformer-based architectures, achieve outstanding classification performance by employing millions of trainable parameters and deep hierarchical feature extraction mechanisms [2]. Although these models produce excellent predictive accuracy, they demand powerful Graphics Processing Units (GPUs), substantial memory capacity, and high computational throughput during inference. Such hardware requirements are generally unavailable in the smartphones and portable devices commonly used by farmers, traders, and agricultural workers in rural Bangladesh.

To overcome these hardware constraints, many existing intelligent agricultural applications rely on cloud-based inference services. While cloud computing effectively removes local computational limitations, it introduces several practical problems. First, prediction latency increases because every inference requires image transmission, server-side processing, and network communication. Second, continuous internet connectivity becomes a mandatory requirement, making these systems unreliable in rural orchards where network coverage is

inconsistent or unavailable. Third, repeated data transmission increases operational costs while raising concerns regarding data privacy, scalability, and long-term service availability [3].

Varieties such as langra and Kanchon Langra, or Bari-4 and Nilambori, possesses subtle differences in shape, surface texture, and peel characteristics that are difficult to distinguish using conventional computer vision techniques or manual inspection alone. These fine-grained visual similarities require models capable of extracting discriminative feature representations while maintaining computational efficiency suitable for resource-constrained deployment [4]. Existing studies frequently emphasize maximizing classification accuracy without adequately considering whether the resulting models can realistically operate on commodity smartphones under practical field conditions.



Another important limitation is the lack of comprehensive development-oriented model evaluation. Many previous studies compare architectures primarily using classification accuracy while overlooking equally important engineering metrics such as inference latency, memory footprint, storage requirements, and deployment feasibility. Selecting a model solely because it

achieves the highest accuracy may not result in the most suitable solution for real-world applications. A model that is marginally more accurate but significantly larger and computationally expensive may perform poorly on mobile hardware, leading to slower response times, higher energy consumption, and reduced usability [5]. Furthermore, although TensorFlow Lite and modern mobile AI frameworks have made on-device inference increasingly accessible, relatively few studies have demonstrated a complete end-to-end deployment pipeline specifically designed for indigenous Bangladeshi mango classification. Existing research often concludes after reporting experimental accuracy obtained in desktop environments without validating whether the trained model can be successfully optimized, integrated into a mobile application, and executed entirely offline under real deployment conditions.

Addressing these limitations requires more than simply identifying the most accurate CNN architecture. Instead, it necessitates a systematic evaluation of multiple lightweight convolutional neural networks under identical experimental conditions to determine which architecture provides the best trade-off between recognition accuracy and deployment efficiency.

Five lightweight transfer learning-based CNN architectures—**MobileNetV2**, **MobileNetV3-Small**, **EfficientNetV2B0**, **DenseNet121**, and **NASNetMobile**—are benchmarked using identical training and evaluation protocols. The comparison incorporates multiple deployment-oriented performance indicators, including classification accuracy, precision, recall, F1-score, model storage size, and inference latency. Based on this multi-objective evaluation, the selected model is further optimized through **TensorFlow Lite Float16 post-training quantization** and deployed within a Flutter-based Android application, **AmmChini AI**.

### 1.3 RESEARCH OBJECTIVES

The primary objective of this research is to develop an efficient, lightweight, and fully offline Edge Artificial Intelligence (Edge AI) framework for the automated classification of indigenous Bangladeshi mango varieties using deep learning. Unlike conventional studies that emphasize classification accuracy alone, this research aims to identify a practical solution capable of operating on resource-constrained mobile devices while maintaining high predictive performance, low computational complexity, and real-time inference. The proposed framework integrates model benchmarking, deployment-oriented optimization, and mobile application development into a unified system suitable for precision agriculture applications.

To accomplish this goal, the following specific research objectives have been formulated:

### **1. To benchmark multiple lightweight convolutional neural network architectures for mango variety classification.**

This study evaluates five state-of-the-art transfer learning-based CNN architectures—**MobileNetV2, MobileNetV3-Small, EfficientNetV2B0, DenseNet121, and NASNetMobile**—using the publicly available **Mangifera2012** dataset. All models are trained under identical experimental conditions to ensure a fair and unbiased comparison. The benchmarking process considers not only classification accuracy but also precision, recall, F1-score, model size, and inference latency to determine the most suitable architecture for mobile edge deployment.

### **2. To investigate architecture-specific preprocessing strategies for robust model training.**

Different convolutional neural network architectures require different input preprocessing procedures to achieve optimal performance. This research implements an architecture-aware preprocessing pipeline that automatically applies the appropriate image normalization and preprocessing function corresponding to each CNN model. This ensures consistent training conditions, prevents preprocessing mismatches, and improves the reproducibility and reliability of the benchmarking process.

### **3. To identify the most suitable lightweight CNN architecture for resource-constrained edge devices.**

Rather than selecting a model solely based on its classification accuracy, this study performs a multi-objective evaluation considering predictive performance alongside deployment-oriented metrics such as computational efficiency, storage requirements, and inference speed. The objective is to determine the architecture that provides the most effective balance between recognition accuracy and real-world deployability on Android smartphones.

### **4. To optimize the selected deep learning model for mobile deployment using TensorFlow Lite.**

After selecting the most appropriate architecture, the trained model is converted into **TensorFlow Lite (TFLite)** format and optimized using **Float16 post-training quantization**. This optimization aims to reduce the model's storage footprint while preserving classification performance, thereby enabling efficient execution on mobile devices with limited computational resources and memory capacity.

### **5. To develop a fully functional Android-based Edge AI application (AmmChini AI).**

A user-friendly mobile application named **AmmChini AI** is developed using the Flutter framework. The application integrates the optimized TensorFlow Lite model and allows users to capture mango images directly using the smartphone camera or select images from the gallery. The system performs entirely offline inference and instantly displays the predicted mango variety along with its confidence score, eliminating dependence on cloud computing or internet connectivity.

## CHAPTER 2

### LITERATURE REVIEW

#### 2.1 GLOBAL RESEARCH ON MANGO CLASSIFICATION

The application of Artificial Intelligence (AI) and Computer Vision in agriculture has experienced remarkable growth over the past decade, driven by the increasing demand for automated crop monitoring, quality assessment, disease diagnosis, and precision farming technologies [1]. Among various horticultural crops, mango has attracted considerable research attention because of its high commercial value and the complexity associated with manual variety identification.

The emergence of Deep Learning, particularly Convolutional Neural Networks (CNNs), fundamentally transformed agricultural image classification by enabling automatic hierarchical feature learning directly from raw image data. CNNs progressively learn low-level features such as edges and textures before constructing increasingly abstract semantic representations capable of distinguishing fine-grained visual differences among object categories [2].

Global research on mango classification has largely focused on three major application areas: **variety identification**, **ripeness estimation**, and **quality assessment**. Numerous studies have demonstrated that transfer learning using pre-trained CNN architectures significantly improves classification performance while reducing training time and computational cost. By initializing models with weights learned from large-scale datasets such as ImageNet, researchers can effectively adapt general visual features to agricultural classification tasks despite relatively limited domain-specific datasets [3]. This strategy has become the standard practice for fruit classification research due to its strong generalization capability and reduced data requirements.

Beyond maturity and quality assessment, recent research has increasingly explored multi-class mango variety classification. Ratha *et al.* combined **MobileNetV2** as a deep feature extractor with traditional machine learning classifiers to classify **15 Indian mango varieties**. Their comparative experiments demonstrated that the Cubic Support Vector Machine achieved the highest classification accuracy when trained on MobileNetV2 feature embeddings, illustrating the effectiveness of transfer learning for fine-grained fruit recognition. However, the study primarily emphasized predictive accuracy and did not investigate deployment-related factors such as model size, inference latency, or mobile implementation.

Recent investigations have also explored hybrid architectures combining Convolutional Neural Networks with Vision Transformers (ViTs). These approaches aim to leverage the local feature extraction capability of CNNs together with the global contextual modeling ability of transformer architectures. Experimental results indicate that CNN–ViT fusion models can achieve excellent classification accuracy on mango datasets while improving feature robustness under diverse imaging conditions. Nevertheless, these architectures generally introduce substantially higher



computational complexity and memory requirements, limiting their suitability for resource-constrained edge devices despite their impressive predictive performance.

Parallel developments have focused on lightweight neural network architectures specifically designed for mobile computing environments. MobileNet, ShuffleNet, GhostNet, and EfficientNet families significantly reduce computational complexity through architectural innovations such as depthwise separable convolution, inverted residual blocks, squeeze-and-excitation modules, and compound scaling strategies [8]. These lightweight architectures achieve competitive recognition performance while substantially reducing parameter count, floating-point operations (FLOPs), memory consumption, and inference latency. As a result, they have become increasingly attractive for agricultural applications requiring deployment on smartphones, embedded processors, drones, and Internet of Things (IoT) devices.

Among lightweight architectures, the MobileNet family has received particular attention because it was specifically designed for mobile and embedded vision applications. MobileNetV2 introduced inverted residual blocks and linear bottlenecks to improve computational efficiency without sacrificing representational capability, while MobileNetV3 further incorporated Neural Architecture Search (NAS), squeeze-and-excitation attention mechanisms, and hardware-aware optimization to improve the trade-off between classification accuracy and inference speed. These characteristics make MobileNet architectures particularly suitable for real-time Edge AI applications where limited computational resources constrain deployment possibilities.

Despite substantial progress in deep learning-based mango classification, several important limitations remain evident throughout the global literature. First, many studies primarily evaluate predictive performance using classification accuracy while overlooking deployment-oriented performance indicators such as model size, inference latency, energy consumption, and memory requirements. From an engineering perspective, these metrics are equally important because an accurate model that cannot execute efficiently on mobile hardware provides limited practical value for field deployment. Second, the majority of existing datasets concentrate on internationally popular commercial cultivars such as Kent, Alphonso, Tommy Atkins, or Aiwen, whereas relatively little attention has been devoted to indigenous South Asian mango varieties exhibiting much higher inter-class visual similarity [9]. Finally, although numerous publications demonstrate impressive laboratory results, comparatively few validate their proposed models through complete end-to-end mobile deployment capable of performing fully offline inference under realistic agricultural conditions.

These observations highlight a significant gap between academic algorithm development and practical agricultural implementation. Existing literature has convincingly demonstrated that CNN-based approaches outperform conventional machine learning techniques for mango image classification. However, comparatively limited research has simultaneously addressed **classification accuracy, computational efficiency, model optimization, and real-world mobile deployment** within a unified Edge AI framework. Bridging this gap is essential for translating

advances in computer vision into intelligent agricultural systems that are both scientifically robust and practically deployable.

The present research addresses these limitations by systematically benchmarking five lightweight transfer learning-based convolutional neural networks—**MobileNetV2**, **MobileNetV3-Small**, **EfficientNetV2B0**, **DenseNet121**, and **NASNetMobile**—using a unified experimental protocol. Unlike many previous studies, model selection is based not solely on predictive accuracy but on a comprehensive evaluation incorporating classification performance, weighted F1-score, inference latency, storage requirements, and deployment feasibility. Furthermore, the selected architecture is optimized using **TensorFlow Lite Float16 post-training quantization** and integrated into the **AmmChini AI** mobile application, demonstrating a complete end-to-end Edge AI solution for offline classification of indigenous Bangladeshi mango varieties. This deployment-oriented perspective differentiates the proposed research from much of the existing global literature and contributes toward the practical adoption of deep learning in precision agriculture.

## 2.2 LIGHTWEIGHT CNNs IN AGRICULTURE

The rapid advancement of Artificial Intelligence (AI) has transformed modern agriculture by enabling intelligent systems capable of automating complex tasks such as crop monitoring, disease diagnosis, fruit classification, yield estimation, and quality assessment. Most agricultural operations are conducted in open fields, orchards, and rural regions where access to high-performance computing infrastructure is limited. Consequently, there has been a growing shift from developing highly complex deep neural networks toward designing lightweight Convolutional Neural Networks (CNNs) that can perform accurate inference directly on resource-constrained edge devices such as smartphones, drones, and embedded systems [1].

Lightweight CNNs are specifically designed to minimize computational complexity while preserving competitive classification performance. Lightweight networks employ architectural optimizations that significantly reduce memory consumption and computational cost. These optimizations include **depthwise separable convolutions**, **inverted residual connections**, **linear bottlenecks**, **channel attention mechanisms**, and **hardware-aware neural architecture search (NAS)**. As a result, lightweight CNNs require fewer floating-point operations (FLOPs), occupy less storage, consume less energy, and achieve lower inference latency, making them particularly suitable for deployment on mobile and embedded platforms [2].

One of the earliest and most influential lightweight architectures is **MobileNet**. MobileNet replaces conventional convolution operations with **depthwise separable convolutions**, reducing computational complexity by decomposing a standard convolution into two simpler operations: a depthwise convolution that independently filters each input channel and a pointwise convolution that combines information across channels [3]. Mathematically, the computational cost of a standard convolution can be expressed as

$$[\text{Cost}_{\text{standard}} = D_K^2 \times M \times N \times D_F^2]$$

where:

- $D_K$  = kernel dimension,
- $M$  = number of input channels,
- $N$  = number of output channels,
- $D_F$  = spatial dimension of the feature map.

In contrast, depthwise separable convolution reduces the computational cost to

$$[\text{Cost}_{\text{depthwise}} = D_K^2 \times M \times D_F^2 + M \times N \times D_F^2]$$

Building upon this foundation, **MobileNetV2** introduced **inverted residual blocks** and **linear bottlenecks**, addressing the representational limitations of the original MobileNet architecture. Instead of directly compressing feature representations, MobileNetV2 first expands feature dimensions using a lightweight pointwise convolution, performs depthwise convolution, and finally projects the features back into a lower-dimensional bottleneck representation. This inverted residual design improves information flow across the network while preserving computational efficiency [4]. Consequently, MobileNetV2 has become one of the most widely adopted architectures for mobile computer vision applications, including agricultural image classification.

Further improvements were introduced with **MobileNetV3**, which combines hardware-aware Neural Architecture Search (NAS), NetAdapt optimization, squeeze-and-excitation (SE) attention modules, and the computationally efficient **Hard-Swish (H-Swish)** activation function [5]. Unlike manually designed architectures, MobileNetV3 was partially generated using automated architecture search techniques that optimize both recognition performance and execution speed on mobile processors. The inclusion of squeeze-and-excitation blocks enables adaptive channel-wise feature recalibration, allowing the network to emphasize informative visual features while suppressing less relevant responses. These enhancements significantly improve the trade-off between classification accuracy and computational efficiency, making MobileNetV3 particularly suitable for Edge AI applications.

The advantages of lightweight CNNs have made them particularly attractive for agricultural applications where computational resources are inherently limited. Modern precision agriculture increasingly relies on portable devices for real-time decision making, requiring models capable of executing directly on field equipment without cloud dependency. Lightweight CNNs have therefore been successfully employed in numerous agricultural tasks, including plant disease diagnosis, weed detection, fruit maturity estimation, crop classification, livestock monitoring, and automated quality inspection [9]. Their ability to perform rapid inference with low memory requirements enables intelligent agricultural systems to operate reliably even in remote environments where internet connectivity is unavailable.

Several recent studies have demonstrated the practical benefits of lightweight CNNs in fruit classification. MobileNetV2 has been successfully applied to smartphone-based plant disease diagnosis, providing near real-time predictions while consuming significantly fewer computational resources than conventional deep CNNs [10]. GhostNet has been deployed for embedded fruit defect detection, achieving real-time inference on low-power hardware platforms such as Raspberry Pi [11]. Similarly, EfficientNet-Lite models have shown promising performance in crop classification tasks by providing competitive recognition accuracy with reduced computational complexity. These studies collectively indicate that lightweight CNNs can effectively support portable agricultural applications without sacrificing practical usability.

Despite these advances, selecting an appropriate lightweight architecture remains a challenging multi-objective optimization problem. The architecture achieving the highest classification accuracy is not necessarily the most appropriate for deployment on mobile devices. Larger models often require increased storage capacity, higher memory consumption, longer inference times, and greater energy usage, reducing battery life and user responsiveness. Therefore, evaluating lightweight CNNs requires considering multiple performance criteria simultaneously rather than relying exclusively on predictive accuracy. Important deployment-oriented metrics include model size, inference latency, memory footprint, parameter count, computational complexity, and energy efficiency in addition to traditional evaluation measures such as accuracy, precision, recall, and F1-score [12].

The emergence of Edge Artificial Intelligence has further strengthened the importance of lightweight CNN architectures. Edge AI shifts inference from centralized cloud servers to local devices, enabling faster response times, improved privacy, reduced communication costs, and continuous offline operation. However, achieving these benefits depends fundamentally on selecting neural network architectures capable of operating efficiently under the limited computational resources of smartphones and embedded processors. Lightweight CNNs therefore represent the enabling technology that makes practical Edge AI deployment feasible across numerous real-world agricultural scenarios.

The present research builds directly upon these developments by investigating five transfer learning-based CNN architectures—**MobileNetV2, MobileNetV3-Small, EfficientNetV2B0, DenseNet121, and NASNetMobile**—within a unified benchmarking framework. Instead of emphasizing classification accuracy alone, this study evaluates each architecture according to predictive performance, weighted F1-score, inference latency, and model storage requirements. Such a comprehensive evaluation enables the identification of the most deployment-efficient architecture for real-time mango variety classification on Android smartphones. The selected model is subsequently optimized using **TensorFlow Lite Float16 post-training quantization** and integrated into the **AmmChini AI** mobile application, thereby demonstrating a complete Edge AI solution for precision agriculture. This deployment-oriented methodology distinguishes the proposed work from many existing agricultural image classification studies that terminate after laboratory-based performance evaluation without validating practical mobile implementation.

**Table 2.1 Comparison of Popular Lightweight CNN Architectures**

| Architecture             | Key Architectural Innovation            | Major Advantages                                              | Common Agricultural Applications              |
|--------------------------|-----------------------------------------|---------------------------------------------------------------|-----------------------------------------------|
| <b>MobileNetV2</b>       | Inverted Residuals, Linear Bottlenecks  | Low computation, small model size, efficient mobile inference | Plant disease detection, fruit classification |
| <b>MobileNetV3-Small</b> | NAS, Squeeze-and-Excitation, Hard-Swish | Excellent balance between accuracy and latency                | Mobile vision, Edge AI, precision agriculture |
| <b>ShuffleNetV2</b>      | Channel Shuffle, Group Convolution      | Extremely fast inference on mobile CPUs                       | Crop recognition, weed detection              |
| <b>GhostNet</b>          | Ghost Feature Generation                | Very low computational complexity                             | Fruit defect detection, embedded vision       |
| <b>EfficientNet-Lite</b> | Compound Scaling                        | High accuracy with moderate computational cost                | Crop monitoring, plant classification         |

## 2.3 GAPS IN EXISTING STUDIES

Modern Convolutional Neural Networks (CNNs) have consistently outperformed traditional machine learning approaches in tasks such as fruit variety recognition, ripeness estimation, disease detection, and quality assessment [1]. However, despite these technological advancements, a detailed examination of existing research reveals several important limitations that hinder the practical adoption of intelligent mango classification systems in real-world agricultural environments.

One of the most prominent limitations is the widespread use of **computationally intensive deep learning architectures**. Many recent studies employ models such as ResNet, DenseNet, Inception, and EfficientNet because of their superior feature extraction capabilities and high classification accuracy [2]. Although these architectures perform exceptionally well on high-performance computing platforms equipped with Graphics Processing Units (GPUs), they often contain millions of trainable parameters and require substantial memory, processing power, and storage capacity. Such computational demands make them unsuitable for direct deployment on smartphones and embedded devices commonly available in agricultural communities, particularly in developing countries such as Bangladesh. Consequently, many high-performing models remain confined to laboratory environments without practical field implementation.

Another significant research gap is the **limited consideration of deployment-oriented evaluation metrics**. The majority of published studies assess model performance primarily using classification accuracy, precision, recall, and F1-score. While these metrics effectively measure predictive capability, they do not fully reflect the practical requirements of mobile and edge computing applications. In real-world deployment scenarios, factors such as model size, inference latency, memory consumption, computational complexity, and energy efficiency are equally important because they directly influence user experience and device performance. A model that achieves marginally higher classification accuracy may still be unsuitable for mobile deployment

if it requires excessive storage space or produces slow inference on resource-constrained hardware. Therefore, selecting an appropriate model for Edge AI applications requires a balanced evaluation that simultaneously considers both predictive performance and deployment efficiency.

The literature also indicates a **lack of comprehensive benchmarking among lightweight CNN architectures**. Existing studies often evaluate a single model or compare only two architectures using different datasets and experimental settings, making objective comparison difficult. Variations in preprocessing techniques, training strategies, data augmentation methods, and evaluation protocols frequently lead to inconsistent performance comparisons across different publications [3]. As a result, there is limited evidence identifying which lightweight architecture offers the most suitable trade-off between recognition accuracy, computational efficiency, and deployment feasibility for mobile agricultural applications.

A further limitation concerns the **underrepresentation of indigenous Bangladeshi mango varieties** in existing research. Most publicly available datasets and published studies focus on internationally recognized cultivars such as Kent, Alphonso, Tommy Atkins, or Aiwen [4]. While these datasets have contributed substantially to the advancement of fruit classification research, they do not adequately represent the unique diversity of Bangladeshi mango varieties. Indigenous cultivars including Amrapali, Harivanga, Langra, Kanchon Langra, Bari-4, Bari-7, Katimon, Mollika, Fazlee, and Nilambori exhibit subtle morphological differences that create a significantly more challenging fine-grained classification problem. Models trained on international datasets may therefore struggle to generalize effectively when applied to these local varieties.

Another notable gap is the continued **dependence on cloud-based inference systems**. Many intelligent agricultural applications transmit captured images to remote cloud servers for prediction before returning the classification result to the user. Although cloud computing provides virtually unlimited computational resources, it introduces several practical challenges, including network latency, dependence on stable internet connectivity, increased communication costs, and potential privacy concerns [5]. These limitations are particularly problematic in rural agricultural regions where internet connectivity may be unreliable or unavailable. Consequently, cloud-dependent systems often fail to satisfy the real-time operational requirements of farmers and field workers.

Furthermore, existing studies rarely demonstrate a **complete end-to-end deployment pipeline**. In many cases, research concludes after reporting experimental accuracy obtained in desktop environments, without validating whether the trained model can be optimized, converted, and integrated into a practical mobile application. Essential deployment stages—including TensorFlow Lite conversion, model quantization, mobile inference optimization, and user interface integration—are frequently omitted. This creates a gap between algorithm development and practical implementation, limiting the real-world applicability of many proposed solutions.

Based on these observations, it is evident that current research has not fully addressed the combined challenges of **classification accuracy, computational efficiency, deployment**

**optimization, and practical usability** within a single unified framework. There remains a clear need for an integrated Edge AI solution capable of accurately classifying indigenous Bangladeshi mango varieties while simultaneously satisfying the computational constraints of modern mobile devices.

To address these limitations, the present research proposes a comprehensive deployment-oriented framework that differs from previous studies in several important aspects. First, five lightweight transfer learning-based CNN architectures—**MobileNetV2**, **MobileNetV3-Small**, **EfficientNetV2B0**, **DenseNet121**, and **NASNetMobile**—are benchmarked using a unified experimental protocol to ensure fair comparison. Second, model selection is based not only on classification accuracy but also on deployment-related metrics including weighted F1-score, model size, and inference latency. Third, the selected architecture is optimized through **TensorFlow Lite (TFLite) Float16 post-training quantization** to reduce storage requirements while maintaining predictive performance. Finally, the optimized model is integrated into the **AmmChini AI** Android application, enabling fully offline, real-time mango variety classification directly on smartphones without requiring internet connectivity. By combining comparative benchmarking, deployment optimization, and mobile implementation within a single framework, this research aims to bridge the gap between academic deep learning research and practical precision agriculture applications.

**Table 2.2 Research Gaps Identified from Existing Literature**

| Research Gap                                    | Observation in Existing Studies                                            | How This Research Addresses the Gap                                                  |
|-------------------------------------------------|----------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Computationally expensive deep learning models  | High-performing CNNs often require GPUs and cloud infrastructure           | Evaluates lightweight CNNs suitable for resource-constrained mobile devices          |
| Accuracy-focused evaluation                     | Most studies prioritize classification accuracy over deployment efficiency | Considers accuracy, weighted F1-score, model size, and inference latency             |
| Limited benchmarking of lightweight models      | Comparisons are generally restricted to one or two architectures           | Benchmarks five lightweight CNN architectures using identical experimental settings  |
| Limited research on Bangladeshi mango varieties | Existing datasets mainly contain international commercial cultivars        | Focuses on ten indigenous Bangladeshi mango varieties                                |
| Dependence on cloud-based inference             | Most systems require internet connectivity for prediction                  | Develops a fully offline Edge AI solution using TensorFlow Lite                      |
| Lack of end-to-end deployment                   | Research often ends after model training and evaluation                    | Implements complete deployment from benchmarking to Flutter-based mobile application |

## 2.4 SUMMARY OF RESEARCH GAP

The literature reviewed in the preceding sections demonstrates that deep learning has substantially advanced the field of automated mango classification. Recent studies have shown that Convolutional Neural

Networks (CNNs) consistently outperform traditional machine learning approaches by automatically learning discriminative visual features from image data, resulting in significant improvements in classification accuracy [1]. Furthermore, transfer learning has become the preferred strategy for agricultural image classification because it enables pre-trained models to achieve high predictive performance even when domain-specific datasets are relatively small. These developments clearly establish deep learning as an effective solution for fruit variety recognition and precision agriculture applications.

Despite these advancements, the existing body of research reveals a noticeable gap between algorithmic development and practical deployment. Most previous studies prioritize predictive accuracy while paying comparatively little attention to deployment-oriented considerations such as computational efficiency, model size, inference latency, and hardware constraints. As a result, many high-performing models require substantial computational resources and are primarily evaluated on desktop workstations or cloud-based infrastructures. Although these architectures perform exceptionally well in laboratory environments, they often fail to satisfy the practical requirements of real-time inference on resource-constrained mobile devices commonly used in agricultural field operations.

Another important limitation identified in the literature is the insufficient emphasis on comprehensive lightweight model benchmarking. Existing studies frequently evaluate only one or two deep learning architectures, making it difficult to objectively determine which model offers the best balance between predictive performance and deployment efficiency. Moreover, many published comparisons rely exclusively on accuracy-based evaluation while overlooking equally important engineering metrics such as model storage size, inference latency, memory consumption, and mobile execution performance. For Edge Artificial Intelligence (Edge AI) applications, these deployment-oriented metrics are critical because they directly influence user experience, battery consumption, responsiveness, and scalability on portable devices.

The review also highlights the limited research conducted on indigenous Bangladeshi mango varieties. Most internationally available datasets and published studies focus on globally commercial cultivars such as Kent, Alphonso, Aiwen, or Tommy Atkins, whereas relatively few investigations consider the unique morphological characteristics of Bangladeshi mangoes. Indigenous cultivars including Amrapali, Harivanga, Langra, Kanchon Langra, Bari-4, Bari-7, Katimon, Mollika, Fazlee, and Nilambori exhibit subtle visual differences that create a challenging fine-grained image classification problem. Consequently, there remains a need for research specifically addressing lightweight deep learning models capable of accurately recognizing these local varieties while maintaining practical deployment feasibility.

Furthermore, although TensorFlow Lite and modern mobile AI frameworks have significantly simplified on-device inference, comparatively few studies present a complete end-to-end deployment pipeline extending beyond model training. Many investigations conclude after reporting experimental performance metrics without demonstrating whether the trained model can be optimized, converted, integrated into a mobile application, and executed entirely offline under real-world conditions. This disconnect between algorithm development and practical implementation limits the real-world impact of many existing agricultural AI solutions.



Based on these observations, the present research is designed to address the identified gaps through a comprehensive deployment-oriented Edge AI framework. Unlike previous studies, this work systematically benchmarks **five lightweight transfer learning-based convolutional neural networks—MobileNetV2, MobileNetV3-Small, EfficientNetV2B0, DenseNet121, and NASNetMobile—using a unified experimental protocol**. Rather than selecting the best architecture solely according to classification accuracy, the proposed framework performs a multi-objective evaluation incorporating **accuracy, precision, recall, weighted F1-score, model storage size, and inference latency**. This balanced evaluation enables the identification of the most suitable architecture for deployment on resource-constrained Android devices.

After selecting the optimal model, the research further extends beyond conventional benchmarking by applying **TensorFlow Lite (TFLite) Float16 post-training quantization**, reducing storage requirements while preserving predictive performance for efficient mobile inference. Finally, the optimized model is integrated into **AmmChini AI**, a Flutter-based Android application capable of performing **fully offline, real-time mango variety classification** directly on the user's smartphone without requiring cloud connectivity. By combining comparative benchmarking, deployment-aware optimization, and practical mobile implementation within a single framework, this research bridges the gap between academic deep learning research and real-world agricultural applications.

**Table 2.3 Summary of Identified Research Gaps and Contributions**

| <b>Research Gap</b>                         | <b>Observation from Existing Studies</b>                                    | <b>Contribution of This Research</b>                                              |
|---------------------------------------------|-----------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Focus on predictive accuracy                | Most studies prioritize classification accuracy over deployment feasibility | Evaluates both predictive performance and deployment-oriented metrics             |
| Limited lightweight model comparison        | Comparisons usually involve only one or two CNN architectures               | Benchmarks five lightweight CNN models under identical experimental conditions    |
| Insufficient deployment evaluation          | Model size, latency, and mobile execution are often ignored                 | Includes model size, inference latency, and Edge AI performance evaluation        |
| Cloud-dependent inference                   | Most intelligent systems rely on internet connectivity                      | Develops a fully offline TensorFlow Lite-based mobile solution                    |
| Limited work on Bangladeshi mango varieties | Existing research primarily targets international commercial cultivars      | Focuses on ten indigenous Bangladeshi mango varieties                             |
| Lack of end-to-end implementation           | Research often ends after model training                                    | Proposes a complete pipeline from benchmarking to Flutter-based mobile deployment |

# CHAPTER 3

## BACKGROUND KNOWLEDGE

### 3.1 MACHINE LEARNING

Machine Learning enables computers to learn from data and improve their performance without being explicitly programmed. Instead of relying on predefined rules, machine learning algorithms identify patterns and relations within datasets to make decisions or predictions. By learning from past experiences, these algorithms can adapt to new data and solve complex problems efficiently. The term **Machine Learning** was introduced by Arthur Samuel in 1959, who described it as the field of study that gives computers the ability to learn without being explicitly programmed. Since then, machine learning has evolved rapidly due to availability of large datasets, powerful computing resources, and advanced learning algorithms. Today, it serves as the foundation for many intelligent systems capable of analyzing complex data and making accurate predictions [1].

The primary objective of machine learning is to develop mathematical problems that can learn patterns from historical data and generalize this knowledge to unseen data. During the training process, the model analyzes input data and adjusts its internal parameters to minimize prediction errors. Once trained, the model can be used to classify data, predict future outcomes, or identify hidden relationships. The performance of a machine learning model depends on the quality of training data, feature representation and the learning algorithm used. Machine learning algorithms are generally classified into three categories: supervised learning, unsupervised learning, and reinforcement learning.

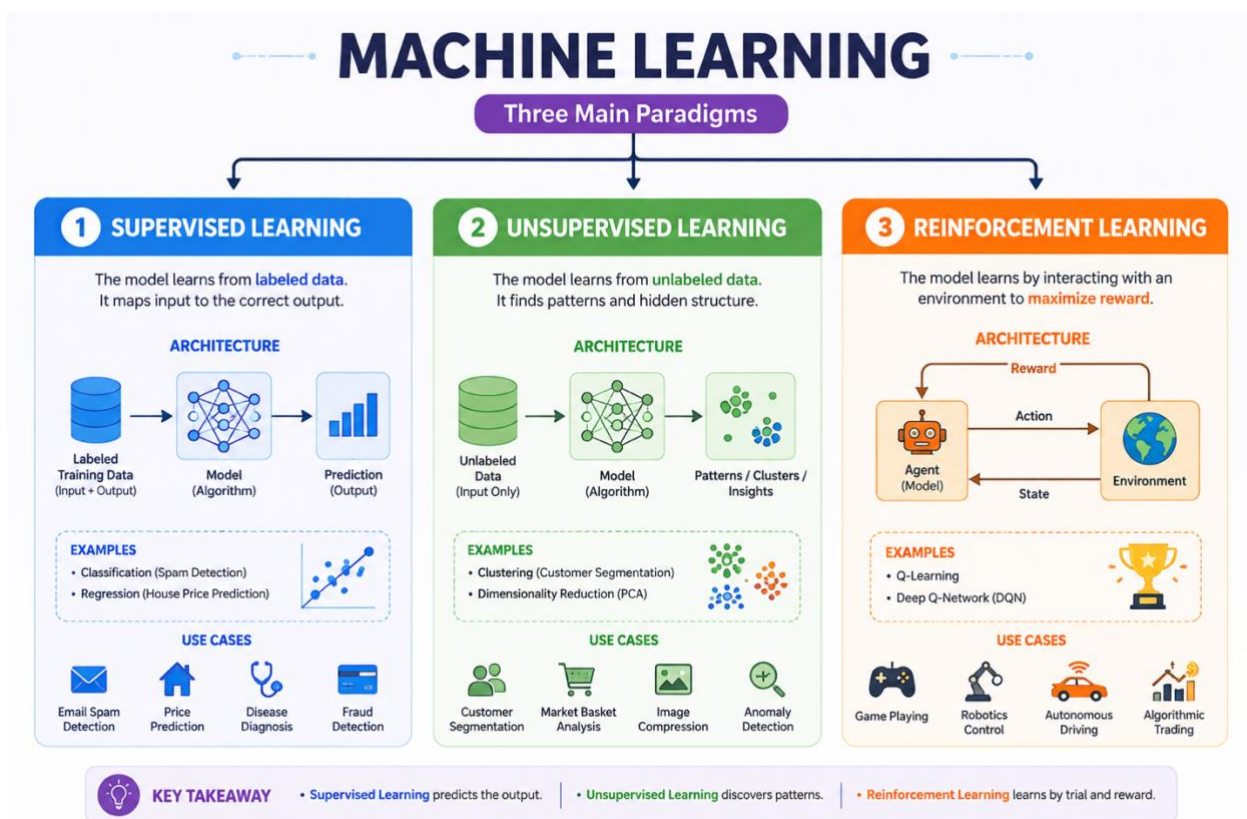
**Supervised learning** is the most widely used machine learning approach. In supervised learning, the model is trained using labeled data, where each input sample is associated with a known output or target label. The objective is to learn the relationship between the input features and the corresponding outputs so that the model can accurately predict the labels of unseen data. Supervised learning is commonly used for classification and regression problems. Algorithms such as Logistic Regression, Decision Trees, Random Forests, Support Vector Machines (SVM), k-Nearest Neighbors (KNN), and Neural Networks are examples of supervised learning techniques. Since this research uses labeled images of Bangladeshi mango varieties, it falls under the category of supervised learning.

**Unsupervised learning** is used when the training data do not contain predefined labels. Instead of predicting known outputs, the algorithm attempts to discover hidden patterns, structures, or relationships within the data. Clustering and dimensionality reduction are common unsupervised learning tasks. Popular algorithms include k-Means Clustering, Hierarchical Clustering, and Principal Component Analysis (PCA). Unsupervised learning is useful for exploratory data

analysis, customer segmentation, anomaly detection, and pattern discovery but is generally not suitable for image classification tasks where class labels are available.

**Reinforcement learning** is a learning paradigm in which an intelligent agent interacts with its environment by performing actions and receiving rewards or penalties. The objective is to learn an optimal strategy, known as a policy, that maximizes cumulative rewards over time. Unlike supervised learning, reinforcement learning does not rely on labeled datasets but instead learns through trial and error. This approach has been successfully applied in robotics, autonomous navigation, game playing, and resource optimization. However, reinforcement learning is not commonly used for static image classification problems.

Machine Learning has become increasingly important in agriculture due to its ability to analyze large amount of agricultural data and support intelligent decision making. It is used in crop disease detection, yield prediction, weed identification, soil analysis, irrigation management, and fruit quality assessment. By automating these tasks, machine learning reduces manual effort, improves accuracy, and enhances agricultural productivity.



## 3.2 Deep Learning

Deep Learning is a specialized branch of machine learning that uses artificial neural networks with multiple hidden layers to learn complex patterns from large amount of data. Deep learning automatically learns hierarchical features representations directly from raw data. This capability has made deep learning one of the most effective approaches for solving tasks such as image

classification, object detection, speech recognition, natural language processing, and medical image analysis.

The rapid advancement of deep learning has been driven by three major factors: the availability of large datasets, increased computational power provided by Graphics Processing Units (GPUs), and the development of advanced neural network architectures. These factors have enabled deep learning models to achieve remarkable performance in various computer vision and artificial intelligence applications. Today, deep learning is widely adopted in both academic research and industry due to its ability to learn highly discriminative features and produce accurate predictions.

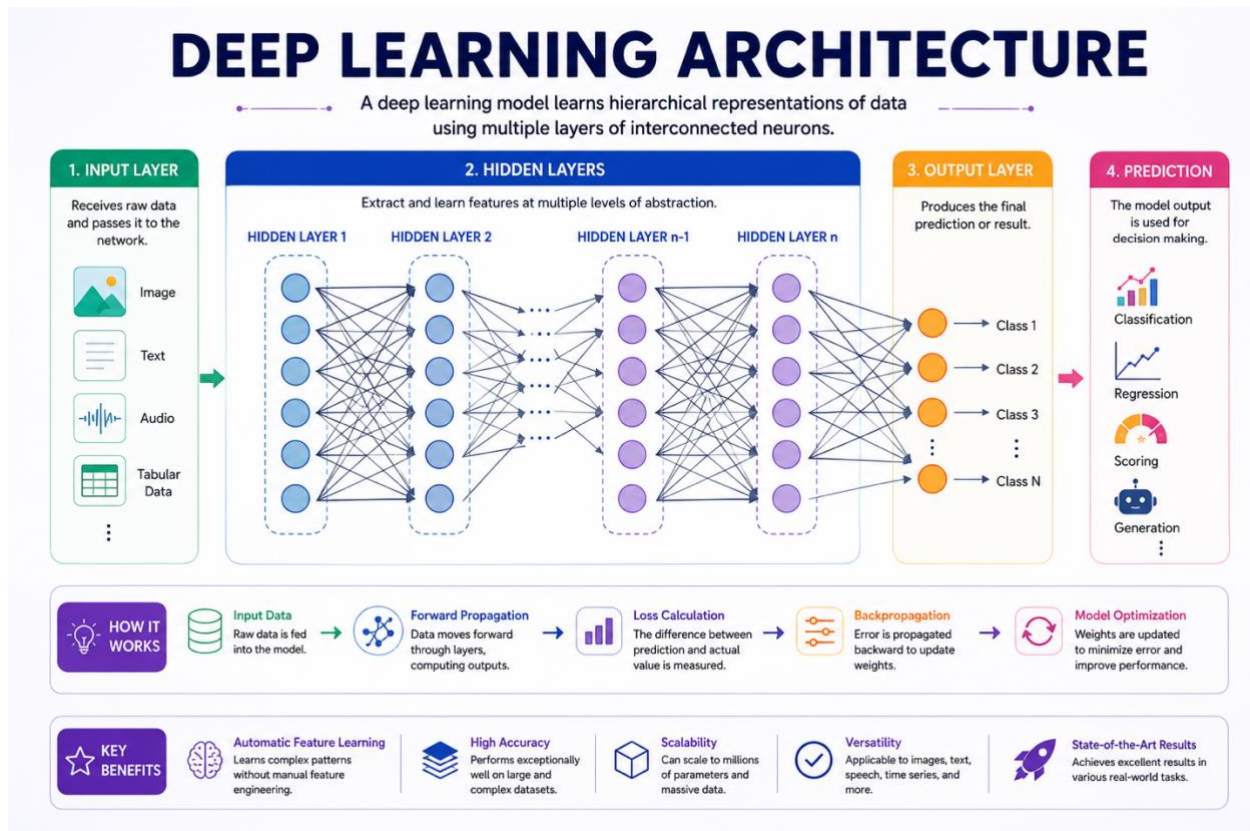
Deep learning models are built upon **Artificial Neural Networks (ANNs)**, which are computational models inspired by the structure and functioning of the human brain. An ANN consists of interconnected processing units called **neurons**, organized into three main layers: the input layer, one or more hidden layers, and the output layer. The input layer receives the input data, the hidden layers perform feature extraction and pattern learning, and the output layer generates the final prediction or classification result. During the training process, the network adjusts its internal weights and biases through optimization algorithms such as gradient descent to minimize prediction errors.

A key advantage of deep learning over traditional machine learning is its ability to perform **automatic features extraction**. Conventional machine learning techniques require manually designed features, such as color, texture, shape, or edge descriptors, before classification. The effectiveness of these methods largely depends on the quality of the handcrafted features. In contrast, deep learning models automatically learn relevant features directly from the input data. Lower layers learn simple features such as edges and textures, while deeper layers gradually capture more complex patterns and object-specific characteristics. This hierarchical learning process significantly improves classification performance, particularly for complex image recognition tasks.

Deep learning has become dominant approach in **computer vision**, where images often contain complex visual patterns that are difficult to represent using handcrafted features. Deep neural networks can effectively recognize objects despite variations in lighting conditions, viewing angles, image backgrounds, scale, and orientation. As a result, deep learning has been successfully applied to facial recognition, autonomous driving, medical diagnosis, industrial quality inspection, and agricultural image analysis.

In agriculture, deep learning has enabled the development of intelligent systems for crop disease detection, fruit classification, weed recognition, plant species identification, yield estimation, and quality assessment. By analyzing digital images, deep learning models can accurately identify subtle visual differences that may be difficult for humans to distinguish. This capability improves productivity, reduces manual inspection, and supports precision agriculture.

Among various deep learning architectures, **Convolutional Neural Networks (CNNs)** have become the most widely used models for image classification tasks. CNNs are specifically designed to process image data by automatically extracting spatial features through convolution operations. Compared with traditional neural networks, CNNs require fewer parameters, achieve higher accuracy, and are more efficient for visual recognition problems. Because of these advantages, CNNs have become the standard architecture for many computer vision applications.



### 3.3 Convolutional Neural Network (CNN)

Convolutional Neural Networks (CNNs) are a specialized class of deep learning models designed to process and analyze image data. Inspired by the organization of the visual cortex in the human brain, CNNs automatically learn hierarchical features from images, eliminating the need for manual feature extraction. Due to their exceptional ability to recognize visual patterns, CNNs have become the most widely used deep learning architecture for computer vision tasks such as image classification, object detection, image segmentation, facial recognition, and medical image analysis.

Traditional image classification methods rely on handcrafted features such as color histograms, texture descriptors, and shape characteristics. The performance of these methods largely depends on the quality of the manually extracted features, which often fail to generalize under varying

lighting conditions, backgrounds, and viewing angles. In contrast, CNNs automatically extract meaningful features directly from raw images during the training process. This automatic feature learning capability enables CNNs to achieve significantly higher classification accuracy than conventional machine learning methods.

A typical CNN consists of several interconnected layers, each performing a specific function in the feature extraction and classification process. The main components include the input layer, convolutional layers, activation layers, pooling layers, fully connected layers, and the output layer.

### INPUT LAYER

The input layer receives the image that will be processed by the network. Before being fed into the model, images are usually resized to a fixed resolution and normalized to ensure consistent input dimensions. Each image is represented as a matrix of pixel values, where color images contain three channels corresponding to red (R), green (G), and blue (B). Proper preprocessing improves training stability and model performance.

### CONVOLUTIONAL LAYER

The convolutional layer is the core component of a CNN. It extracts important visual features from the input image using small matrices called **filters** or **kernels**. During convolution, each filter slides across the image and performs element-wise multiplication followed by summation to generate a **feature map**. Different filters learn different image characteristics, such as edges, corners, textures, and shapes.

The convolution operation can be expressed as:

$$F(i, j) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} I(i + m, j + n) K(m, n)$$

where:

- $F(i, j)$  = Output feature map at position  $(i, j)$
- $I(i + m, j + n)$  = Input image pixel value
- $K(m, n)$  = Convolution kernel (filter)
- $M$  = Height of the kernel
- $N$  = Width of the kernel
- $m$  and  $n$  = Kernel indices

As training progresses, the network automatically learns the optimal filter values that best distinguish different image classes.

### ACTIVATION FUNCTION

After each convolution operation, an activation function is applied to introduce non-linearity into the network. Without non-linear activation, the network would behave like a simple linear model and would be unable to learn complex image patterns.

The most commonly used activation function is the **Rectified Linear Unit (ReLU)**, defined as

$$\text{ReLU}(x) = \max(0, x)$$

ReLU replaces negative values with zero while keeping positive values unchanged. Compared with traditional activation functions such as Sigmoid and Tanh, ReLU is computationally efficient, accelerates training, and helps reduce the vanishing gradient problem.

### **POOLING LAYER**

The Pooling Layer reduces the spatial dimensions of feature maps while preserving the most important information. This process decreases computational complexity, reduces memory requirements, and helps prevent overfitting.

The two most common pooling methods are:

**Max Pooling:** Selects the maximum value within each pooling window.

**Average Pooling:** Computes the average value within each pooling window.

Among these methods, max pooling is more widely used because it preserves the strongest and most informative features extracted by the convolutional layers.

### **FULLY CONNECTED LAYER**

After several convolutional and pooling operations, the extracted feature maps are flattened into a one-dimensional feature vector. This vector is then passed to one or more fully connected layers, where the learned features are combined to perform classification.

The fully connected layers act similarly to traditional artificial neural networks by learning relationships between high-level image features and their corresponding class labels.

### **OUTPUT LAYER**

The final layer of a CNN produces the prediction result. For multi-class classification problems, the **Softmax** activation function is commonly used because it converts the output values into probabilities that sum to one. The predicted class is the one with the highest probability.

The Softmax function is defined as

$$P(y_i) = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}$$

where



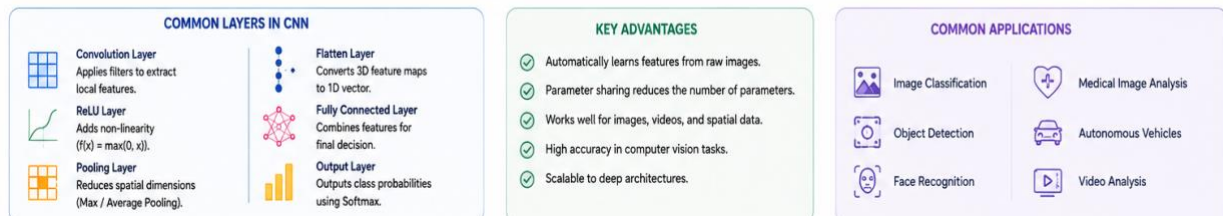
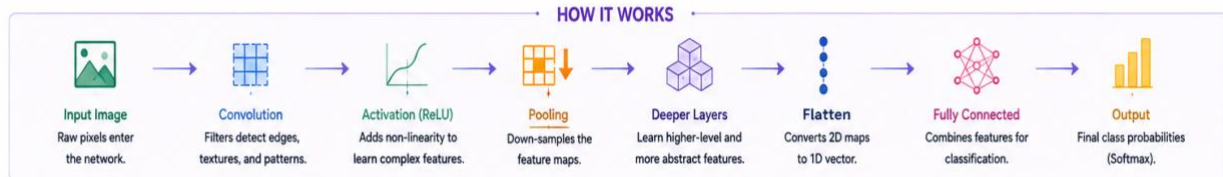
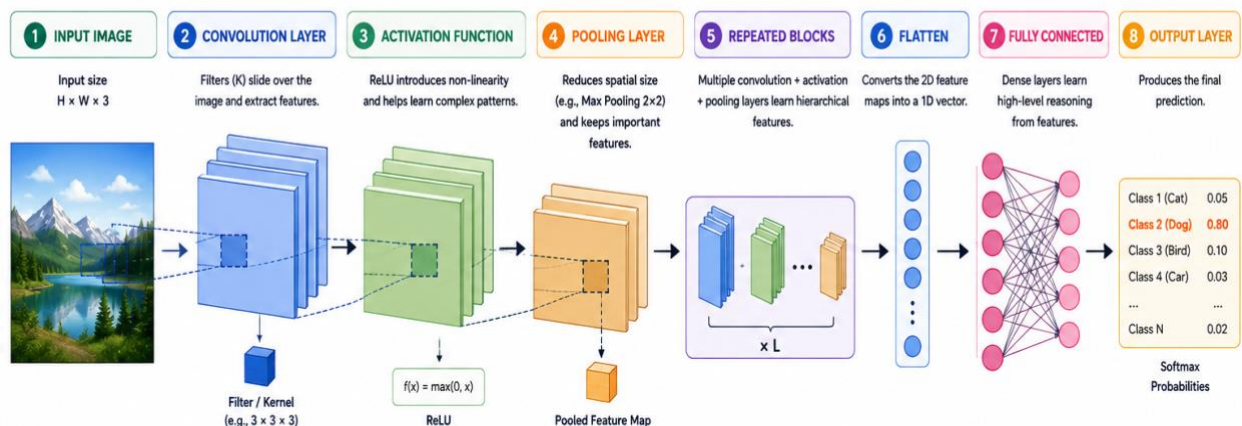
- $P(y_i)$  = the predicted probability of the  $i$ th class
- $z_i$  = output score (logit) of the  $i$ th class before applying the Softmax function
- $e$  = Euler's number (approximately 2.71828)
- $C$  = the total number of classes
- $\sum_{j=1}^C e^{z_j}$  = sum of the exponential values of the output scores for all classes

## Training Process of a CNN

The training of a CNN involves two major stages: **forward propagation** and **backpropagation**. During forward propagation, the input image passes through the network, and a prediction is generated. The prediction is compared with the actual label using a loss function, such as categorical cross-entropy. The resulting error is then propagated backward through the network using the backpropagation algorithm. During this process, the weights of the network are updated using optimization algorithms such as Stochastic Gradient Descent (SGD) or Adam. This iterative learning process continues until the model achieves satisfactory performance on the training data.

## CONVOLUTIONAL NEURAL NETWORK (CNN) ARCHITECTURE

— CNNs are powerful deep learning models for analyzing visual data. —





### 3.4 Edge Artificial Intelligence (Edge AI)

Edge Artificial Intelligence (Edge AI) is a computing paradigm that combines artificial intelligence with edge computing to enable AI models to perform data processing and inference directly on edge devices, such as smartphones, embedded systems, Internet of Things (IoT) devices, drones, and wearable devices. Unlike traditional cloud-based AI systems, where data must be transmitted to remote servers for processing, Edge AI performs computations locally on the device. This approach reduces dependency on internet connectivity while enabling faster and more secure decision-making.

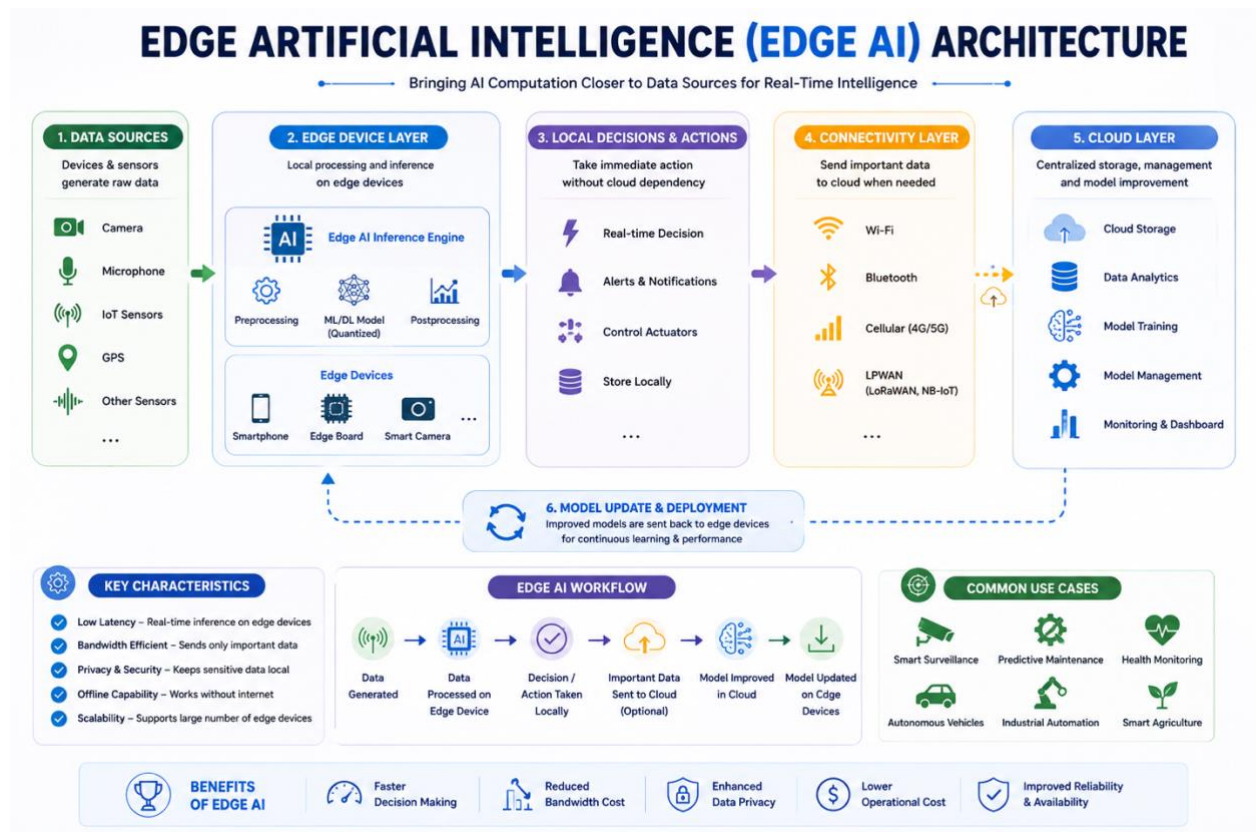
The rapid growth of Internet of Things (IoT) devices and the increasing demand for real time intelligent applications have accelerated the development of Edge AI. Advances in lightweight deep learning models, mobile processors, and AI acceleration hardware have made it possible to deploy complex neural networks on resource-constrained devices. As a result, Edge AI has become an important technology in applications requiring low latency, enhanced privacy, and continuous offline operation.

In a conventional cloud-based AI system, data collected from a user device are transmitted to a cloud server, where the trained model performs inference and returns the predicted results. Although cloud computing provides high computational power and large storage capacity, it introduces several limitations, including communication delays, dependence on stable internet connectivity, increased bandwidth usage, and potential privacy concerns due to the transmission of sensitive data. These limitations can significantly affect applications that require immediate responses or operate in remote areas with unreliable network coverage.

Edge AI addresses these challenges by executing AI models directly on local devices. Since data are processed locally, there is no need to send images or sensor data to external servers. This significantly reduces inference latency, minimizes network traffic, and improves data privacy. Local processing also enables intelligent systems to continue operating even without an internet connection, making Edge AI particularly suitable for real-time and mission-critical applications.

Several advantages make Edge AI an attractive solution for modern intelligent systems. First, it provides **low latency** by eliminating the communication delay associated with cloud-based processing. Second, **offline functionality** allows AI applications to operate without continuous internet access. Third, **enhanced privacy and security** are achieved because sensitive data remain on the user's device rather than being transmitted to external servers. Fourth, **reduced bandwidth consumption** lowers communication costs and improves system efficiency. Finally, Edge AI offers improved reliability, as predictions can still be generated even when network connectivity is unavailable.

Despite these advantages, deploying AI models on edge devices presents several challenges. Smartphones and embedded systems generally have limited processing power, memory capacity, storage space, and battery life compared to cloud servers. Large deep learning models with millions of parameters require substantial computational resources and are therefore unsuitable for direct deployment on such devices. To overcome these limitations, researchers have developed lightweight neural network architectures, including MobileNet, EfficientNet, and NASNet, which significantly reduce model size and computational complexity while maintaining high classification accuracy.



### 3.5 TensorFlow Lite

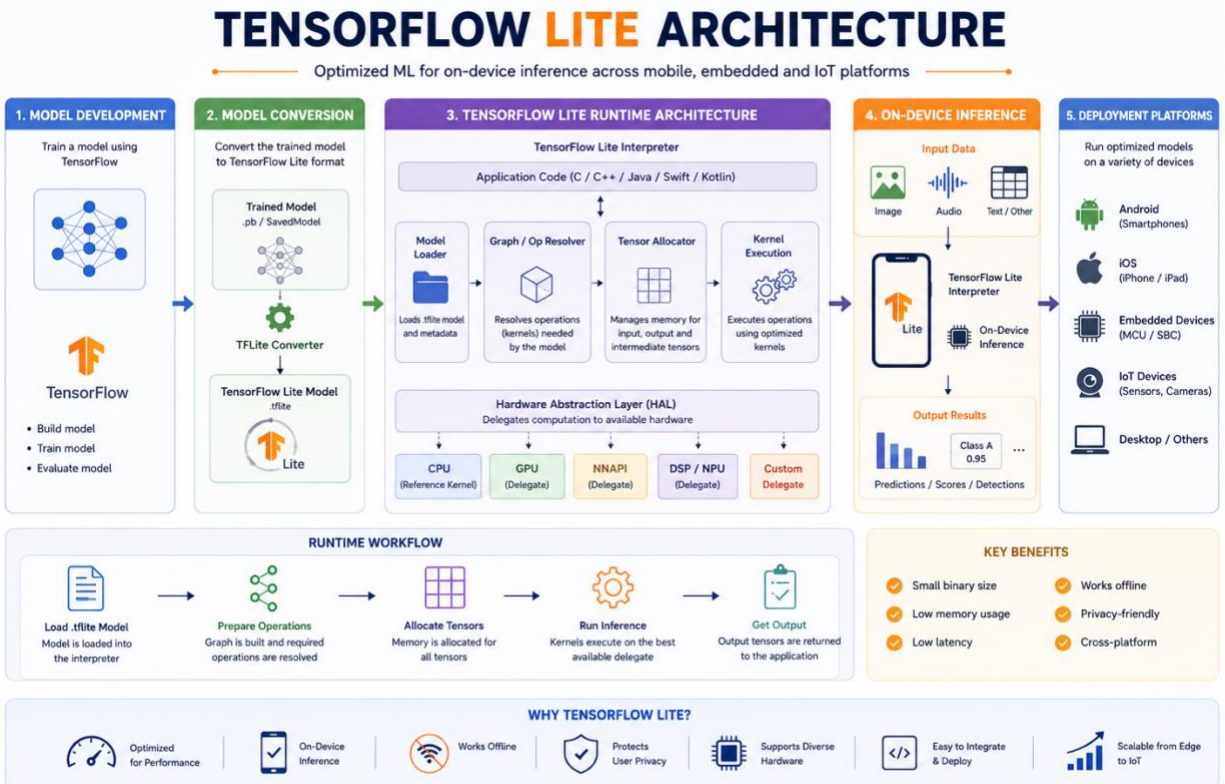
TensorFlow Lite is a lightweight version of Tensorflow framework developed by google for deploying machine learning models on mobile and emedded devices. It is specially designed to perform on-device inference efficiently while minimizing memory usage, computational requirements, and power consumption. TensorFlow Lite enables AI models to run directly on edge devices without relying on cloud servers, making it an important technology for Edge AI applications.

Unlike the standard TensorFlow framework, which is manually used for training deep learning models, TensorFlow Lite is optimized for model deployment. It uses a compact **.tflite** model format that allows faster inference and lower storage requirements. The deployment process involves training a model in TensorFlow, converting it into TensorFlow Lite model using TensorFlow Lite Converter, and integrating the converted model into a mobile or embedded application.

TensorFlow also supports several optimization techniques, such as **quantization**, which reduces the model size and improves inference speed. Among these techniques, **Float16 quantization** converts model weights from 32-bit floating-point values to 16-bit values, reducing memory consumption while maintaining high classification accuracy. These optimizations make TensorFlow Lite suitable for devices with limited computational resources.

TensorFlow Lite is widely used in applications such as image classification, object detection, speech recognition, and smart healthcare systems. Its ability to perform fast and online inference makes it particularly useful for mobile applications that require real-time predictions.

In this research, TensorFlow Lite is used to deploy the trained lightweight CNN model on an android smartphone. The selected model is converted into the **.tflite** format and integrated into the Flutter-based mobile application. This enables real-time classification of Bangladeshi mango varieties directly on the device without requiring an internet connection, resulting in low latency, improved privacy, and efficient performance on mobile edge devices.



# CHAPTER 4

## METHODOLOGY

### 4.1 Model Architecture Selection

Selecting an appropriate CNN architecture is essential for developing an image classification system that can operate efficiently on resource-constrained edge devices. While many deep learning models achieve excellent recognition accuracy, their computational complexity often limits their use in real-time mobile applications. Therefore, this study emphasizes both predictive performance and deployment efficiency rather than accuracy alone.

To identify the most suitable architecture for offline Android deployment, five transfer learning-based CNN models were selected for comparison:

- **MobileNetV2**
- **MobileNetV3-Small**
- **EfficientNetV2B0**
- **DenseNet121**
- **NASNetMobile**

These architectures represent different lightweight CNN design approaches and are fully compatible with **TensorFlow Lite**. Each model also provides pretrained **ImageNet** weights, making them well suited for transfer learning.

Instead of training from scratch, all models were initialized with pretrained ImageNet weights to leverage previously learned low-level visual features such as edges, textures, and color patterns. This approach reduces training time, accelerates convergence, and improves generalization, particularly when working with a relatively small dataset.

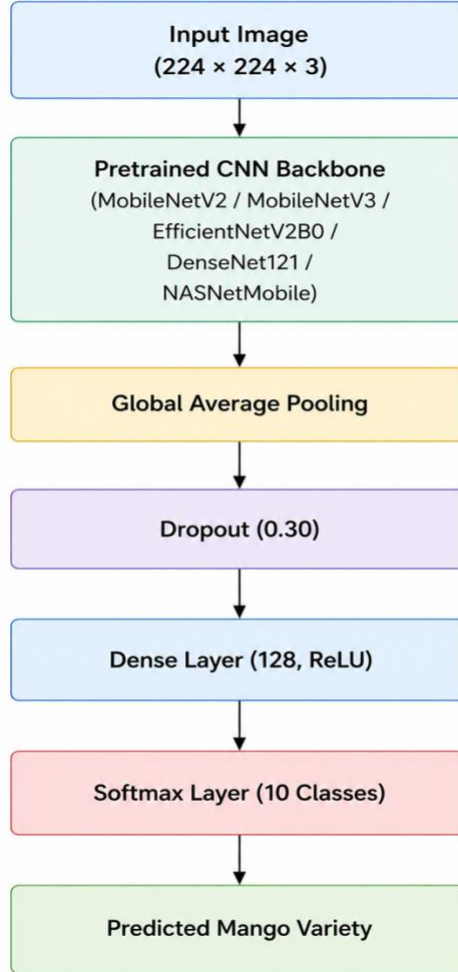
To ensure a fair comparison, every architecture followed the same transfer learning strategy. The original classification layer was removed (`include_top=False`), and the pretrained backbone remained frozen during the initial training stage while only the newly added classifier was optimized. This preserves the learned feature representations and minimizes overfitting.

An identical classification head was attached to each backbone, consisting of:

- **Global Average Pooling (GAP)**
- **Dropout (0.30)**
- **Dense layer (128 neurons, ReLU)**
- **Softmax output layer (10 classes)**

The GAP layer reduces feature dimensionality with fewer trainable parameters than conventional fully connected layers, while Dropout improves model generalization by reducing overfitting. Finally, the Softmax layer generates the probability distribution for the ten indigenous Bangladeshi mango varieties, ensuring a consistent evaluation across all architectures.

The architecture of the classifier can be summarized as follows:



## 4.2 Mathematical Representation

Let

$$(I \in R^{224 \times 224 \times 3})$$

represent an input RGB image.

The pretrained CNN backbone extracts feature representations

$$[F = f_{\theta}(I)]$$

where

- “I” is the resized input image,
- $(f_{\theta})$  denotes the convolutional feature extractor,
- “F” represents the extracted feature map.

Global Average Pooling converts the feature map into a feature vector

$$[z = \text{GAP}(F)]$$

which is then passed through the dense classifier

$$h = \text{ReLU}(W_1 z + b_1)$$

Finally, the Softmax function computes the posterior probability of each mango variety

$$P(y = i|I) = \frac{e^{h_i}}{\sum_{j=1}^{10} e^{h_j}}$$

where

- $(P(y = i|I))$  represents the probability that the input image belongs to class  $\setminus(i\setminus)$ ,
- $(i = 1, \dots, 10)$ .

The predicted class is determined by

$$[\hat{y} = \arg \max_i P(y = i|I)]$$

### Selection Criteria

Each CNN architecture was evaluated according to the following criteria:

- Classification Accuracy
- Precision
- Recall
- Weighted F1-score
- Model Storage Size (MB)
- Inference Latency (ms)
- Deployment Compatibility with TensorFlow Lite

Unlike conventional image classification studies that emphasize accuracy alone, these criteria collectively assess both predictive capability and practical deployment feasibility. This comprehensive evaluation enables identification of the most suitable architecture for real-time Edge AI applications.

**Table 4.2 Selection Criteria for CNN Benchmarking**

| Evaluation Criterion | Purpose |
|----------------------|---------|
|----------------------|---------|

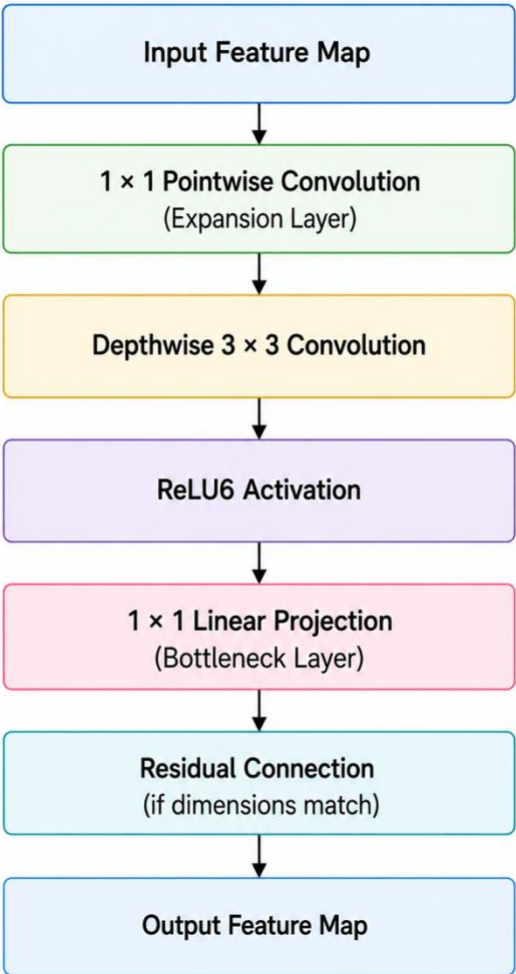
|                      |                                                    |
|----------------------|----------------------------------------------------|
| Accuracy             | Overall classification performance                 |
| Precision            | Reliability of positive predictions                |
| Recall               | Ability to correctly identify true classes         |
| Weighted F1-score    | Balanced evaluation considering class distribution |
| Model Size           | Storage requirement for mobile deployment          |
| Inference Latency    | Real-time execution capability                     |
| TFLite Compatibility | Suitability for Android edge deployment            |

### Architecture Overview

Unlike traditional CNNs that repeatedly increase feature dimensions through standard convolution operations, MobileNetV2 adopts a more computationally efficient design. Each building block consists of three consecutive operations:

1. **Expansion Layer**
2. **Depthwise Convolution**
3. **Linear Projection (Bottleneck)**

The overall workflow of a MobileNetV2 residual block is illustrated below.



### Depthwise Separable Convolution

The computational efficiency of MobileNetV2 primarily originates from **depthwise separable convolution**, which replaces computationally expensive standard convolution operations.

A standard convolution simultaneously performs spatial filtering and channel combination using

$$[D_K^2 \times M \times N]$$

filters, where

- $D_K$  = kernel size,
- $M$  = number of input channels,
- $N$  = number of output channels.

The computational complexity is therefore

$$[Cost_{standard} = D_K^2 \times M \times N \times D_F^2]$$

where  $D_K$  denotes the spatial resolution of the feature map.

MobileNetV2 decomposes this operation into two independent stages:

#### Depthwise Convolution

Each input channel is filtered independently using a single convolution kernel.

The computational cost becomes

$$[D_K^2 \times M \times D_F^2]$$

#### Pointwise Convolution

A subsequent  $(1 \times 1)$  convolution combines information across channels.

Its computational cost is

$$[M \times N \times D_F^2]$$

Therefore, the total computational complexity becomes

$$[Cost_{Depthwise} = D_K^2 M D_F^2 + M N D_F^2]$$

which is considerably smaller than standard convolution.

For typical CNN configurations, depthwise separable convolution reduces computation by nearly **8–9 times**, enabling efficient execution on mobile processors without substantially compromising feature extraction capability [1].



### 4.3 MobileNetV3-Small

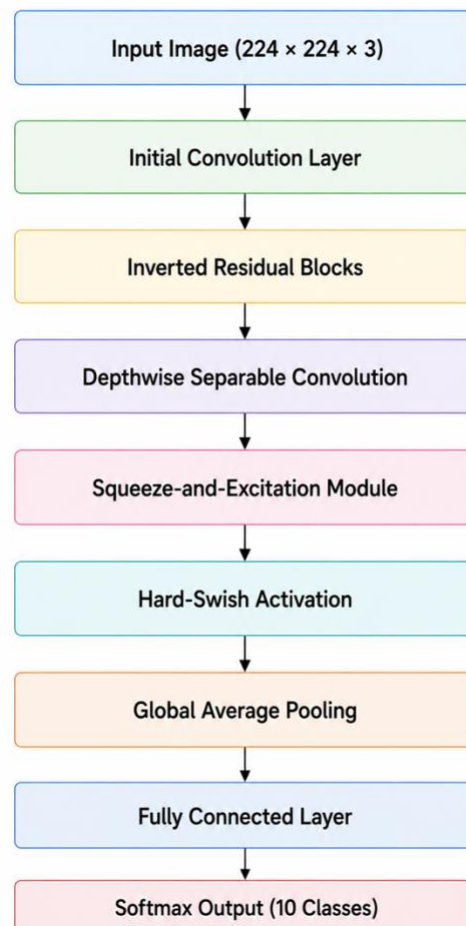
MobileNetV3-Small is the primary deep learning architecture adopted in this research for deployment on mobile edge devices. Developed by Howard *et al.* in 2019, MobileNetV3 represents the third generation of the MobileNet family and was specifically designed to maximize the trade-off between classification accuracy and computational efficiency for resource-constrained hardware [1]. Unlike its predecessors, MobileNetV3 was developed using a combination of **Neural Architecture Search (NAS)** and **NetAdapt**, enabling the network architecture to be automatically optimized for both predictive performance and execution speed on mobile processors.

#### Architecture Overview

MobileNetV3-Small inherits the efficient **depthwise separable convolution** mechanism from previous MobileNet architectures while introducing several additional optimizations, including:

- Inverted Residual Blocks
- Depthwise Separable Convolution
- Squeeze-and-Excitation (SE) Attention Modules
- Hard-Swish (H-Swish) Activation Function
- Hardware-aware Neural Architecture Search (NAS)

The overall architecture can be conceptually represented as follows.



**Figure 3.5:** Simplified architecture of MobileNetV3-Small used in this research.

### Depthwise Separable Convolution

Similar to MobileNetV2, MobileNetV3 employs **depthwise separable convolution** to reduce computational complexity.

Instead of simultaneously learning spatial and channel relationships through standard convolution, the operation is divided into:

- Depthwise Convolution
- Pointwise Convolution

This decomposition substantially decreases the number of floating-point operations (FLOPs) while maintaining effective feature extraction.

The computational complexity becomes

$$[Cost_{DW} = D_K^2 M D_F^2 + M N D_F^2]$$

where

- “ $D_K$ ” denotes kernel size,
- “ $M$ ” represents input channels,
- “ $N$ ” represents output channels,
- “ $D_K$ ” denotes feature map dimensions.

Compared with standard convolution, depthwise separable convolution dramatically reduces computational requirements, making MobileNetV3 highly suitable for mobile inference.

### Inverted Residual Block

MobileNetV3 retains the **Inverted Residual Block** introduced in MobileNetV2.

Rather than connecting expanded feature representations, shortcut connections operate between compact bottleneck layers.

The residual mapping is defined as

$$[y = x + F(x)]$$

where

- “ $x$ ” represents the bottleneck input,
- “ $F(x)$ ” denotes the transformed feature representation.

This design improves gradient propagation while minimizing memory consumption.

### Squeeze-and-Excitation Attention Module

One of the most important improvements introduced in MobileNetV3 is the incorporation of the **Squeeze-and-Excitation (SE) module**.

Traditional convolution layers treat every feature channel equally. However, not all feature channels contribute equally to image recognition. The SE module enables the network to learn which channels are most informative by assigning adaptive importance weights during training.

The SE module operates in three stages:

### 1. Squeeze

Global Average Pooling compresses spatial information into a channel descriptor.

$$[z_c = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W x_c(i, j)]$$

where

- $H$  and  $W$  denote feature map height and width,
- $(x_c(i, j))$  is the activation of channel  $c$ .

### 2. Excitation

The channel descriptor passes through two fully connected layers to generate channel-wise attention weights.

$$[s = \sigma(W_2 \delta(W_1 z))]$$

where

- $(\delta(\cdot))$  is the ReLU activation,
- $(\sigma(\cdot))$  is the Sigmoid function.

### 3. Feature Recalibration

Each feature channel is multiplied by its corresponding attention weight.

$$[\hat{x}_c = s_c x_c]$$

This mechanism enables MobileNetV3 to emphasize discriminative visual features while suppressing less informative responses, improving classification performance for fine-grained image recognition tasks such as mango variety classification.

## 4.4 EfficientNetV2B0

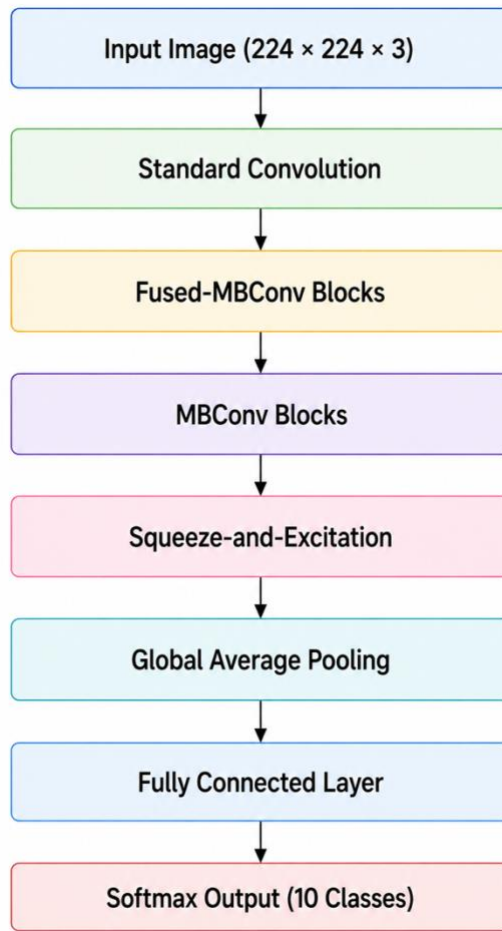
EfficientNetV2B0 is a modern convolutional neural network architecture designed to achieve high classification accuracy while maintaining improved computational efficiency compared with earlier deep learning models. Proposed by Tan and Le in 2021, EfficientNetV2 was developed to address two major limitations of the original EfficientNet family: slow training speed and high computational cost associated with large-scale image classification tasks [1]. By introducing optimized convolutional blocks and an

improved scaling strategy, EfficientNetV2 achieves faster training, enhanced feature extraction, and competitive classification performance across a wide range of computer vision applications.

### Architecture Overview

EfficientNetV2B0 retains the fundamental philosophy of the EfficientNet family while incorporating several architectural improvements that accelerate training and improve computational efficiency. The network consists of a sequence of convolutional blocks that progressively learn increasingly abstract visual representations from the input image.

The simplified architecture is illustrated below.



**Figure 3.6:** Simplified architecture of EfficientNetV2B0 used in this research.

### Fused-MBConv Block

One of the most significant innovations introduced in EfficientNetV2 is the **Fused Mobile Inverted Bottleneck Convolution (Fused-MBConv)**.

In the original MBConv block, feature extraction consists of:

- Expansion Layer
- Depthwise Convolution
- Projection Layer

Although computationally efficient during inference, this structure introduces considerable memory access overhead during training.

To overcome this limitation, EfficientNetV2 replaces the early MBConv blocks with Fused-MBConv layers, where the expansion and depthwise convolution are merged into a single standard convolution operation.

This modification improves training efficiency while maintaining strong feature extraction capability.

### MBConv Block

In deeper stages of the network, EfficientNetV2 continues to employ the **Mobile Inverted Bottleneck Convolution (MBConv)** introduced by the EfficientNet architecture.

The MBConv block consists of:

- Pointwise Expansion
- Depthwise Convolution
- Squeeze-and-Excitation Module
- Pointwise Projection

Mathematically,

$$[F(x) = PW \left( DW \left( PW(x) \right) \right)]$$

where

- “PW” denotes pointwise convolution,
- “DW” represents depthwise convolution,
- “F(x)” is the transformed feature representation.

Residual connections are applied whenever the input and output dimensions are identical.

### Compound Scaling Strategy

Unlike conventional CNNs that increase only network depth or width, EfficientNetV2 adopts a **compound scaling strategy** that simultaneously scales multiple architectural dimensions.

The network balances:

- Depth
- Width

- Input Resolution

using scaling coefficients that maximize recognition performance under computational constraints.

The scaling process is represented as

$$[\text{Depth} = \alpha^\phi]$$

$$[\text{Width} = \beta^\phi]$$

$$[\text{Resolution} = \gamma^\phi]$$

subject to

$$[\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2]$$

where

- $(\phi)$  denotes the compound scaling coefficient,
- $(\alpha, \beta, \gamma)$  are constants determined during architecture optimization.

This balanced scaling strategy allows EfficientNetV2 to achieve high recognition accuracy without excessively increasing computational complexity.

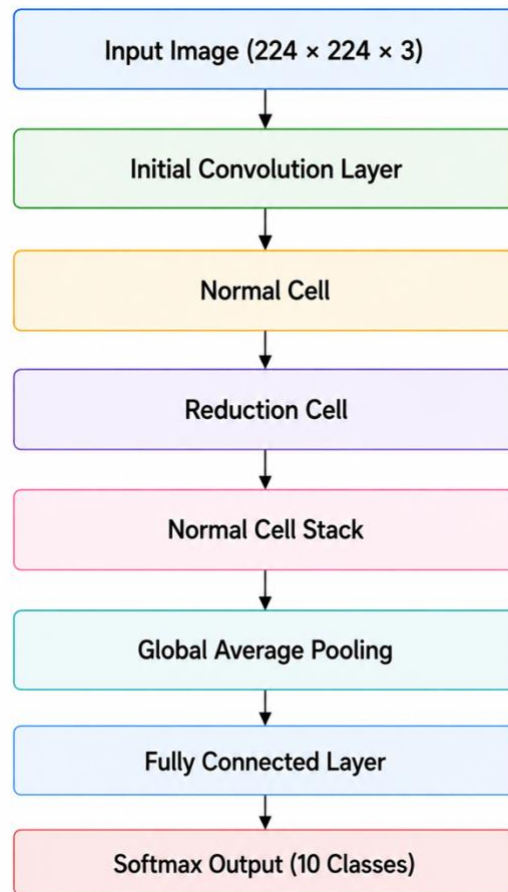
#### 4.5 NASNetMobile

NASNetMobile is a lightweight convolutional neural network architecture developed to achieve high classification performance while maintaining computational efficiency for mobile and embedded devices. Unlike conventional CNN architectures that are manually designed by researchers, NASNet (Neural Architecture Search Network) was automatically generated using **Neural Architecture Search (NAS)**, an Artificial Intelligence-based optimization technique that explores thousands of possible network configurations to identify the architecture that provides the best balance between predictive accuracy and computational cost [1]. This automated design methodology enables NASNetMobile to achieve competitive recognition performance while reducing the need for manual architecture engineering.

#### Architecture Overview

The distinguishing feature of NASNetMobile is that its architecture was not manually constructed but automatically discovered through a reinforcement learning-based Neural Architecture Search process. Instead of designing every convolutional layer individually, NAS learns two reusable computational building blocks known as the **Normal Cell** and the **Reduction Cell**. These cells are repeatedly stacked to construct the complete neural network.

The simplified architecture is illustrated below.



### Normal Cell

The **Normal Cell** is responsible for extracting high-level visual features while preserving the spatial dimensions of the input feature map. It consists of multiple parallel computational branches that perform different convolutional operations simultaneously. The outputs from these branches are combined to generate a richer feature representation.

The Normal Cell enables the network to learn diverse visual characteristics such as:

- shape,
- texture,
- edges,
- color patterns,
- fine morphological details.

Since the spatial resolution remains unchanged, the Normal Cell focuses primarily on improving feature representation without reducing image size.

## Neural Architecture Search (NAS)

The most important contribution of NASNetMobile is its automated architecture generation process.

Unlike manually engineered networks, Neural Architecture Search formulates architecture design as an optimization problem.

Given an architecture search space

$$[A = \{a_1, a_2, \dots, a_n\}]$$

the search algorithm attempts to identify

$$[a^* = \arg \max_{a \in A} Performance(a)]$$

where

- $(A)$  denotes the candidate architecture space,
- $(Performance(a))$  represents the validation accuracy of architecture  $a$ ,
- $(a^*)$  is the optimal architecture selected by the search process.

The search controller is typically implemented using reinforcement learning, where candidate architectures are evaluated, rewarded according to their performance, and progressively refined until an optimal architecture is obtained.

This automated design strategy enables NASNetMobile to achieve strong predictive performance without relying entirely on manually engineered architectural decisions.

## 4.6 DenseNet121

DenseNet121 is a deep convolutional neural network architecture designed to improve feature propagation, encourage feature reuse, and alleviate the vanishing gradient problem commonly encountered in very deep neural networks. Proposed by Huang *et al.* in 2017, DenseNet (Densely Connected Convolutional Network) introduced a novel connectivity pattern in which every layer is directly connected to all subsequent layers within a dense block [1]. Unlike conventional CNN architectures that pass information only to the immediately following layer, DenseNet establishes direct feature connections between all preceding and succeeding layers. This dense connectivity enables efficient information flow throughout the network while improving learning efficiency and reducing redundant feature extraction.

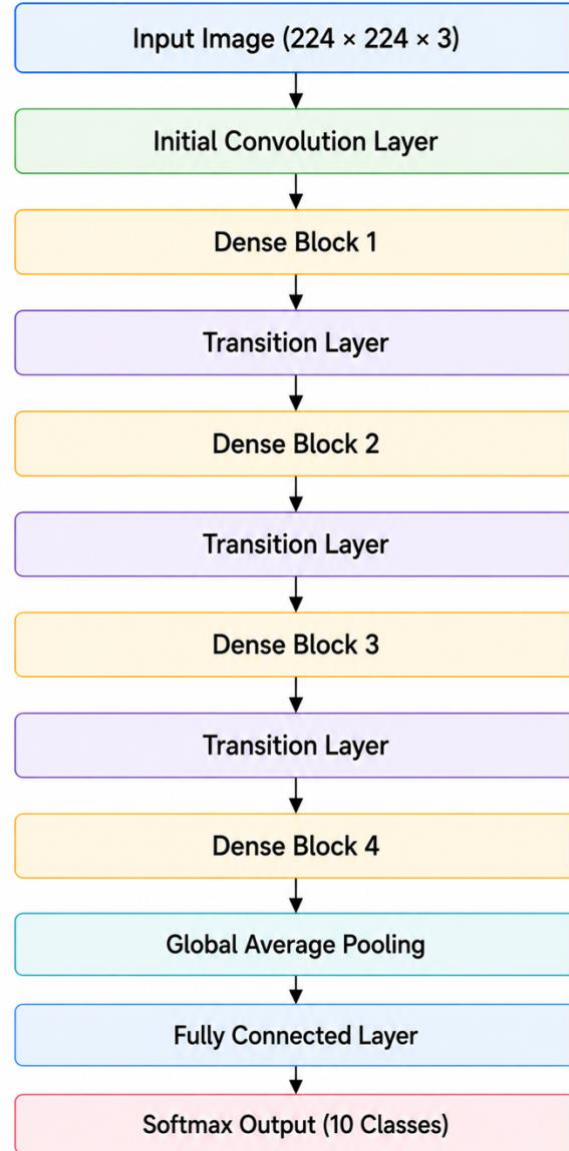
### Architecture Overview

The fundamental concept of DenseNet121 is the **Dense Block**, where each layer receives feature maps generated by every preceding layer. Rather than learning entirely new representations at each layer,



DenseNet continuously reuses previously extracted features, allowing the network to preserve both low-level and high-level visual information throughout the learning process.

The simplified architecture of DenseNet121 is illustrated below.



### Dense Connectivity

Unlike conventional convolutional neural networks where each layer receives input only from its immediate predecessor, DenseNet introduces direct feature propagation across all layers.

For the ( $l^{th}$ ) layer,

$$[x_l = H_l([x_0, x_1, \dots, x_{l-1}])]$$

where

- $(x_l)$  denotes the output of the  $(l^{th})$  layer,
- $(H_l(\cdot))$  represents the sequence of Batch Normalization, ReLU activation, and convolution,
- $([])$  denotes feature map concatenation.

Instead of summing feature maps as performed in ResNet, DenseNet concatenates all previous feature representations.

This dense connectivity offers several important advantages:

- improved gradient propagation,
- efficient feature reuse,
- stronger information preservation,
- reduced feature redundancy,
- improved learning stability.

Consequently, DenseNet121 is capable of learning highly discriminative representations even when trained on relatively small datasets.

### Transition Layer

Without additional processing, repeated feature concatenation would continuously increase the number of feature maps.

To control computational complexity, DenseNet introduces **Transition Layers** between consecutive Dense Blocks.

Each Transition Layer consists of:

- Batch Normalization,
- $(1 \times 1)$  convolution,
- Average Pooling.

These operations reduce both the number of feature channels and the spatial dimensions of the feature maps, thereby limiting memory consumption while preserving the most informative features.

## 4.7 Dynamic Preprocessing Implementation

Deep learning models pretrained on large-scale datasets such as ImageNet are trained using architecture-specific input preprocessing techniques. These preprocessing operations transform raw pixel values into numerical representations that match the data distribution observed during the original pretraining process. Consequently, different convolutional neural network (CNN) architectures expect different input formats before inference. Applying an inappropriate preprocessing function may introduce a distribution mismatch

between the training data and the pretrained feature extractor, resulting in degraded feature learning, slower convergence, and reduced classification accuracy [1].

### Architecture-Specific Preprocessing Functions

The preprocessing functions used in this research are summarized in **Table 4.7**.

**Table 4.7 Architecture-Specific Preprocessing Functions**

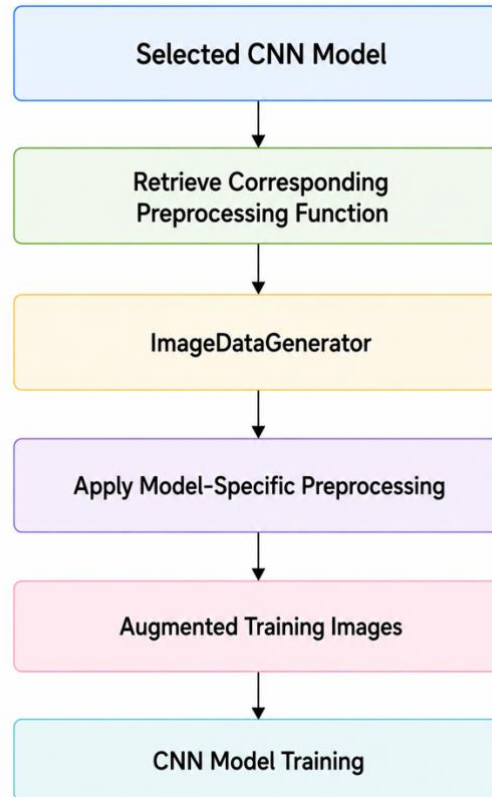
| <b>CNN Architecture</b> | <b>TensorFlow Preprocessing Function</b>        |
|-------------------------|-------------------------------------------------|
| MobileNetV2             | <code>mobilenet_v2.preprocess_input()</code>    |
| MobileNetV3-Small       | <code>mobilenet_v3.preprocess_input()</code>    |
| EfficientNetV2B0        | <code>efficientnet_v2.preprocess_input()</code> |
| DenseNet121             | <code>densenet.preprocess_input()</code>        |
| NASNetMobile            | <code>nasnet.preprocess_input()</code>          |

Each preprocessing function performs the transformations required by its corresponding pretrained architecture. Because these implementations are provided by TensorFlow, they preserve compatibility with the ImageNet feature representations learned during pretraining.

#### 4.8 Dynamic Preprocessing Workflow

Instead of maintaining separate training scripts for each CNN architecture, a unified preprocessing framework was implemented. During execution, the selected CNN architecture determines which preprocessing function is passed to the data generator.

The workflow is illustrated in **Figure 3.9**.



#### Implementation Strategy

The dynamic preprocessing mechanism was implemented by maintaining a preprocessing function associated with each CNN architecture inside the model configuration module. During training, the selected preprocessing function is passed directly to TensorFlow's ImageDataGenerator, which automatically applies the appropriate transformation before generating each batch.

Conceptually, the workflow can be represented as

This modular implementation eliminates the need to manually modify preprocessing code whenever a different CNN architecture is evaluated. Consequently, all benchmark experiments can be executed using the same training pipeline while preserving model-specific preprocessing behavior.

#### Integration with Data Augmentation

An important characteristic of the proposed framework is that dynamic preprocessing is fully integrated with the online data augmentation pipeline described in Section 3.1.2.

During training, each image undergoes the following sequence of operations:

1. Image loading
2. Image resizing ( $224 \times 224$  pixels)
3. Architecture-specific preprocessing
4. Online data augmentation
5. Batch generation
6. CNN training

This integrated workflow ensures that every augmented image also satisfies the preprocessing requirements of its corresponding CNN architecture.

#### 4.9 Architectural Input Disparities

One of the fundamental assumptions of transfer learning is that the input data presented during fine-tuning should follow the same statistical distribution as the data used during the original pretraining stage. Modern convolutional neural networks pretrained on the ImageNet dataset learn feature representations based on architecture-specific preprocessing procedures that transform raw pixel values into normalized numerical distributions. Consequently, different CNN architectures expect different input formats before performing feature extraction. Ignoring these architectural differences may introduce inconsistencies between the input data and the pretrained weights, ultimately reducing feature extraction quality and degrading classification performance [1].

**Table 3.6 Architectural Input Preprocessing Requirements**

| <b>CNN Architecture</b> | <b>Preprocessing Function</b>      | <b>Purpose</b>                                                          |
|-------------------------|------------------------------------|-------------------------------------------------------------------------|
| MobileNetV2             | mobilenet_v2.preprocess_input()    | Prepares images according to MobileNetV2 ImageNet training distribution |
| MobileNetV3-Small       | mobilenet_v3.preprocess_input()    | Matches MobileNetV3-Small pretrained input representation               |
| EfficientNetV2B0        | efficientnet_v2.preprocess_input() | Applies EfficientNetV2-specific preprocessing                           |
| DenseNet121             | densenet.preprocess_input()        | Preserves DenseNet ImageNet normalization scheme                        |

|              |                           |                                                 |
|--------------|---------------------------|-------------------------------------------------|
| NASNetMobile | nasnet.preprocess_input() | Produces NASNet-compatible input representation |
|--------------|---------------------------|-------------------------------------------------|

#### 4.10 Pipeline Design

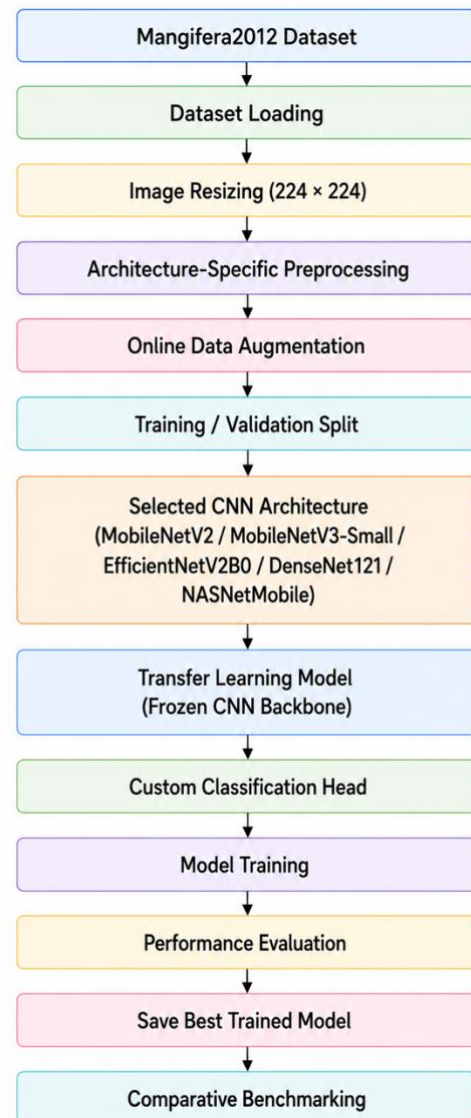
To ensure a fair and reproducible comparison among multiple convolutional neural network (CNN) architectures, this research adopts a unified and modular training pipeline. Instead of developing separate workflows for each model, a single automated pipeline was designed to handle dataset preparation, architecture-specific preprocessing, model construction, training, evaluation, benchmarking, and model storage. This unified implementation minimizes manual intervention, reduces code redundancy, and guarantees that all evaluated models are trained under identical experimental conditions.

#### Overall Pipeline Architecture

The complete pipeline implemented in this research consists of seven major stages:

1. Dataset Loading
2. Image Preprocessing
3. Data Augmentation
4. Model Construction
5. Model Training
6. Performance Evaluation
7. Model Export and Benchmarking

The overall workflow is illustrated in



## 4.10 Training Pipeline

### Stage 1: Dataset Loading

The **Mangifera2012** dataset is loaded directly from its class-wise directory structure, allowing TensorFlow to assign labels automatically based on folder names. This approach ensures consistent label encoding throughout the training process.

### Stage 2: Image Preprocessing

Each image is resized to  $224 \times 224$  pixels before applying the architecture-specific preprocessing function required by the corresponding **ImageNet-pretrained** CNN model.

### Stage 3: Online Data Augmentation

To improve generalization, **ImageDataGenerator** performs online augmentation during training. Random transformations—including rotation, width and height shifts, shear, zoom, and horizontal flipping—generate new image variations in every epoch without increasing the dataset size.

### Stage 4: Model Construction

Each CNN model is initialized with pretrained **ImageNet** weights. The original classification layer is removed, the convolutional backbone remains frozen, and a common classifier is added consisting of:

- Global Average Pooling (GAP)
- Dropout (0.30)
- Dense layer (128 neurons, ReLU)
- Softmax output layer (10 classes)

Using an identical classification head ensures a fair comparison among all architectures.

### Stage 5: Model Training

Training is performed using the **Adam** optimizer with transfer learning. The process incorporates **early stopping**, **best-weight restoration**, and **mini-batch optimization** to improve convergence and reduce overfitting. Training automatically stops when validation performance no longer improves.

### Stage 6: Performance Evaluation

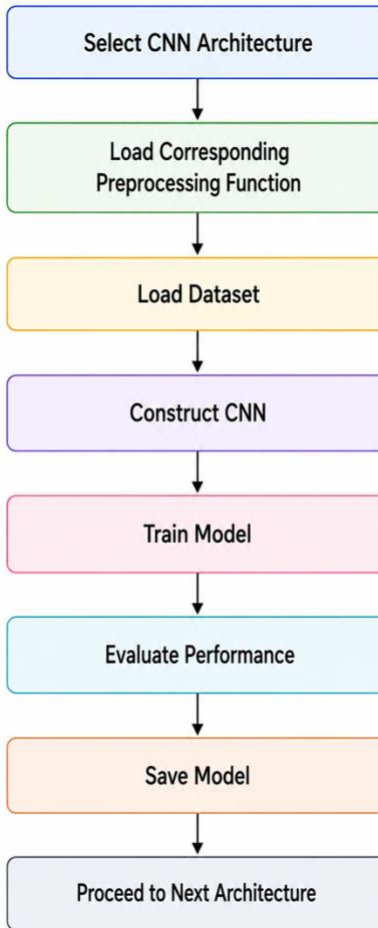
The trained model is evaluated using multiple performance metrics, including **accuracy**, **precision**, **recall**, **weighted F1-score**, **confusion matrix**, and **classification report**, providing a comprehensive assessment of classification performance.

### Stage 7: Model Storage and Benchmarking

After evaluation, the trained model is saved for future deployment. Additional benchmarking measures **model size**, **inference latency**, and **deployment compatibility**. Finally, the results from all five CNN architectures are compared to identify the most suitable model for **Edge AI** deployment.

#### 4.11 Pipeline Automation

An important contribution of this research is the automation of the complete benchmarking process. Rather than manually repeating the training procedure for each CNN architecture, the pipeline automatically performs the following sequence:



This automated workflow significantly improves experimental efficiency while eliminating inconsistencies caused by manual intervention.

#### 4.12 Methodology Flowchart

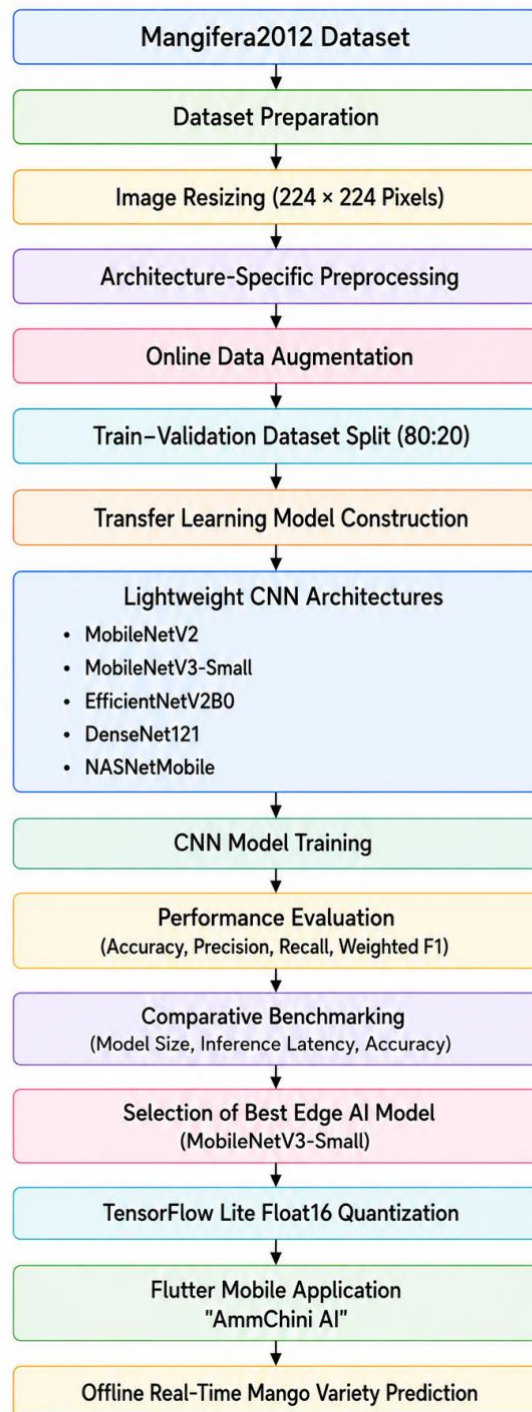
This research follows a structured methodology that integrates dataset preparation, deep learning model development, comparative benchmarking, deployment-oriented optimization, and mobile application implementation into a unified Edge AI framework. Rather than focusing solely on model training, the proposed methodology encompasses the complete lifecycle of an intelligent image classification system, beginning with data preparation and ending with real-time offline deployment on Android smartphones.



The workflow was designed to ensure fairness, reproducibility, and deployment readiness. All evaluated convolutional neural network (CNN) architectures were trained under identical experimental settings using the same dataset, preprocessing pipeline, data augmentation strategy, optimizer configuration, and evaluation protocol. This standardized methodology ensures that differences in experimental performance are attributable to the inherent characteristics of the CNN architectures rather than inconsistencies in implementation.

The complete research methodology adopted in this study is illustrated in **Figure 4.12**.

**Figure 4.12 Proposed Research Methodology Flowchart**



## Description of Methodology Stages

The proposed methodology consists of the following sequential stages:

1. **Dataset Preparation:** The Mangifera2012 dataset containing 2,012 images of ten indigenous Bangladeshi mango varieties is organized for supervised learning.
2. **Image Preprocessing:** All images are resized to **224 × 224 pixels** and processed using architecture-specific preprocessing functions to maintain compatibility with ImageNet-pretrained CNN models.
3. **Data Augmentation:** Online augmentation techniques—including rotation, translation, shear transformation, zooming, and horizontal flipping—are applied to improve model generalization and reduce overfitting.
4. **Transfer Learning:** Five pretrained CNN architectures (**MobileNetV2, MobileNetV3-Small, EfficientNetV2B0, DenseNet121, and NASNetMobile**) are fine-tuned using a common classification head while keeping the convolutional backbone frozen during the initial training stage.
5. **Performance Evaluation:** Each model is evaluated using classification accuracy, precision, recall, weighted F1-score, confusion matrices, and classification reports.
6. **Comparative Benchmarking:** Beyond predictive performance, deployment-oriented metrics such as **model size** and **average inference latency** are measured to identify the most suitable architecture for Edge AI deployment.
7. **Model Optimization:** The selected MobileNetV3-Small model is converted to **TensorFlow Lite (TFLite)** format and optimized using **Float16 post-training quantization**, reducing storage requirements while maintaining classification performance.
8. **Mobile Deployment:** The optimized TFLite model is integrated into the Flutter-based **AmmChini AI** application, enabling fully offline and real-time classification of indigenous Bangladeshi mango varieties on Android smartphones.

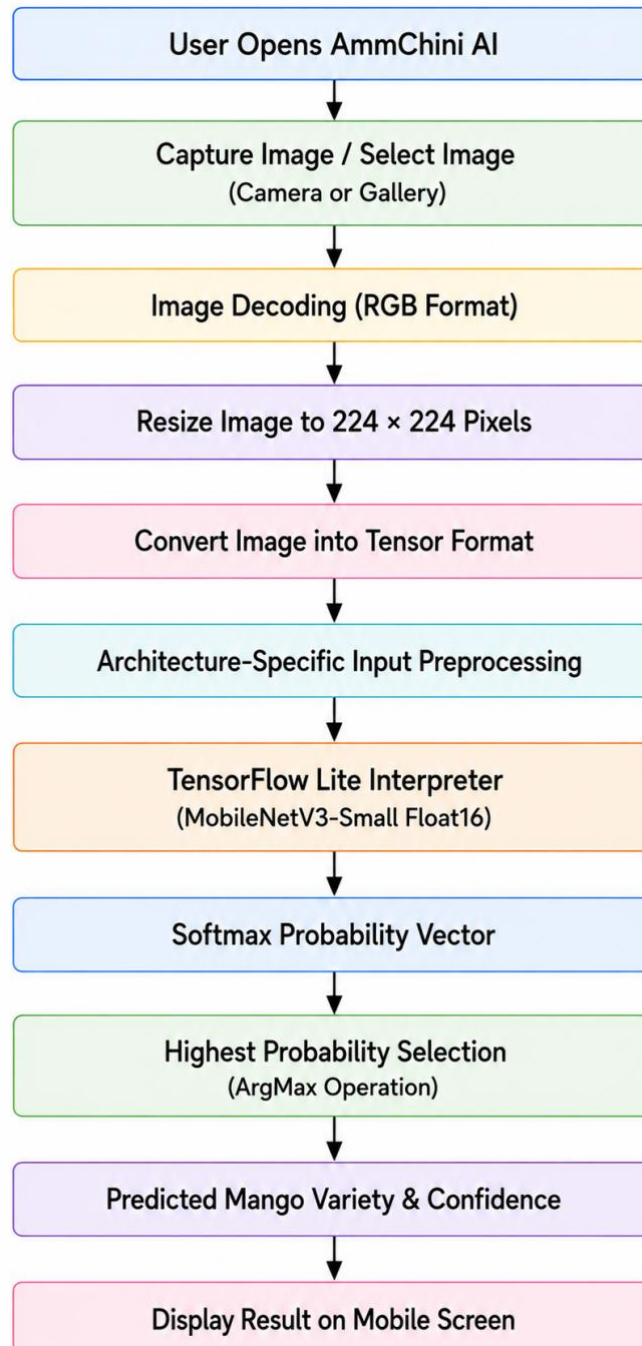
## Importance of the Proposed Methodology

The proposed methodology extends beyond conventional deep learning workflows by combining **comparative model benchmarking, deployment-aware optimization, and mobile application development** within a single unified framework. This end-to-end approach ensures that the final system is not only accurate but also practical for deployment in resource-constrained agricultural environments. By emphasizing computational efficiency alongside predictive performance, the methodology effectively bridges the gap between laboratory-scale computer vision research and real-world precision agriculture applications.

#### 4.13 Operational Workflow

The operational data flow begins when the user launches the AmmChini AI mobile application and selects an image for classification. The application allows users to either capture a new photograph using the smartphone camera or choose an existing image from the device gallery. Once an image is selected, it undergoes a sequence of preprocessing operations before being supplied to the TensorFlow Lite inference engine.

The complete operational workflow is illustrated in **Figure 3.13**.



### 3.5.6 Result Visualization

After inference is completed, the application presents the prediction results through a user-friendly graphical interface.

The output includes:

- Predicted mango variety.
- Classification confidence score.
- Captured image preview.

Because all computations are performed locally, the prediction appears almost instantly, providing a seamless user experience suitable for practical agricultural applications.

**Table 3.7 Operational Stages of the Edge AI Inference Pipeline**

| Stage | Operation                 | Purpose                                      |
|-------|---------------------------|----------------------------------------------|
| 1     | Image Acquisition         | Capture or select mango image                |
| 2     | Image Decoding            | Convert image into RGB format                |
| 3     | Image Resizing            | Resize image to $224 \times 224$ pixels      |
| 4     | Input Preprocessing       | Apply MobileNetV3-specific preprocessing     |
| 5     | Tensor Conversion         | Generate input tensor for inference          |
| 6     | TensorFlow Lite Inference | Execute MobileNetV3-Small model locally      |
| 7     | Softmax Prediction        | Compute probability of each mango variety    |
| 8     | Result Visualization      | Display predicted class and confidence score |

## CHAPTER 5

### TOOLS AND DATASETS

#### 5.1 Dataset Description

The effectiveness of any deep learning-based image classification system is fundamentally dependent on the quality, diversity, and representativeness of the dataset used during model development. A well-constructed dataset enables the neural network to learn robust feature representations, improves generalization capability, and reduces the likelihood of overfitting. Since the objective of this research is to develop an Edge AI system capable of accurately classifying indigenous Bangladeshi mango varieties under practical deployment conditions, it is essential that the training dataset adequately represents the visual diversity encountered in real agricultural environments.

**Table 5.1 Distribution of Mango Varieties in the Mangifera2012 Dataset**

| Class        | Mango Variety     | Number of Images    |
|--------------|-------------------|---------------------|
| 1            | Amrapali          | 252                 |
| 2            | Bari-4            | 235                 |
| 3            | Bari-7            | 176                 |
| 4            | Fazlee            | 156                 |
| 5            | Harivanga         | 202                 |
| 6            | Kanchon Langra    | 210                 |
| 7            | Katimon           | 163                 |
| 8            | Langra            | 202                 |
| 9            | Mollika           | 221                 |
| 10           | Nilambori         | 195                 |
| <b>Total</b> | <b>10 Classes</b> | <b>2,012 Images</b> |

The dataset exhibits a relatively balanced class distribution, with each variety containing between **156 and 252 images**. Although slight variations exist among individual classes, the distribution is sufficiently uniform to minimize significant class imbalance during model training. Maintaining a

balanced dataset is important because highly imbalanced class distributions often bias deep learning models toward majority classes, resulting in poor classification performance for minority categories.

## 5.2 Dataset Composition

The quality and composition of a dataset play a crucial role in determining the performance and generalization capability of deep learning models. In image classification tasks, an appropriately designed dataset should provide sufficient visual diversity while maintaining representative samples for each target category. A dataset containing balanced class distributions and diverse visual characteristics enables convolutional neural networks to learn discriminative feature representations more effectively, thereby improving their robustness when presented with previously unseen images [1]. Consequently, careful examination of the dataset composition is essential before discussing the model training process.

The dataset contains images of the following ten mango varieties:

**Table 3.2 Class-wise Distribution of the Mangifera2012 Dataset**

| Class ID     | Mango Variety     | Number of Images | Percentage (%) |
|--------------|-------------------|------------------|----------------|
| 1            | Amrapali          | 252              | 12.53          |
| 2            | Bari-4            | 235              | 11.68          |
| 3            | Bari-7            | 176              | 8.75           |
| 4            | Fazlee            | 156              | 7.75           |
| 5            | Harivanga         | 202              | 10.04          |
| 6            | Kanchon Langra    | 210              | 10.44          |
| 7            | Katimon           | 163              | 8.10           |
| 8            | Langra            | 202              | 10.04          |
| 9            | Mollika           | 221              | 10.98          |
| 10           | Nilambori         | 195              | 9.69           |
| <b>Total</b> | <b>10 Classes</b> | <b>2,012</b>     | <b>100</b>     |

As shown in **Table 3.2**, the dataset demonstrates a relatively balanced distribution. The largest class, **Amrapali**, contains **252 images**, whereas **Fazlee**, the smallest class, contains **156 images**.

Another important characteristic of the dataset is that every image contains a **single mango instance**, allowing the research to focus exclusively on image classification rather than object detection or instance segmentation. This simplifies the learning objective by assigning exactly one ground-truth label to each input image, enabling the evaluated CNN models to learn direct mappings between image features and mango varieties. Consequently, the task formulation can be expressed mathematically as

$$[f(I) \rightarrow y]$$

where

- $I$  denotes the input mango image,
- $f(.)$  represents the deep convolutional neural network,
- $(y \in \{1,2, \dots, 10\})$  denotes the predicted mango variety.

This formulation defines a standard supervised multi-class classification problem in which the network estimates the posterior probability of each class using the Softmax activation function, and the class with the highest probability is selected as the final prediction.

### 5.3 Data Preprocessing and Augmentation

The quality of input data plays a fundamental role in the performance and generalization capability of deep learning models. Raw images collected from real-world environments often exhibit variations in resolution, orientation, illumination, scale, and viewpoint, which may negatively influence the learning process if not handled appropriately.

#### Image Resizing

The original images in the Mangifera2012 dataset have a native resolution of **3024 × 4032 pixels**, which is considerably larger than the input size required by lightweight convolutional neural networks. Therefore, every input image was resized to **224 × 224 pixels**, which corresponds to the standard input dimension required by **MobileNetV2**, **MobileNetV3-Small**, **EfficientNetV2B0**, **DenseNet121**, and **NASNetMobile**. During resizing, the RGB color channels were preserved, producing input tensors of dimension

$$[ 224 \times 224 \times 3 ]$$

This resizing operation provides an effective balance between computational efficiency and feature preservation.

## 5.6 Architecture-Specific Input Preprocessing

An important aspect of this research is the implementation of **architecture-specific preprocessing**, ensuring that each convolutional neural network receives input data in the format expected by its original ImageNet pre-training configuration.

During implementation, TensorFlow's built-in preprocessing functions were utilized as follows:

- MobileNetV2 → mobilenet\_v2.preprocess\_input()
- MobileNetV3-Small → mobilenet\_v3.preprocess\_input()
- EfficientNetV2B0 → efficientnet\_v2.preprocess\_input()
- DenseNet121 → densenet.preprocess\_input()
- NASNetMobile → nasnet.preprocess\_input()

## 5.7 Data Augmentation

Although the Mangifera2012 dataset provides a balanced representation of indigenous mango varieties, its size remains relatively modest compared with large-scale computer vision datasets. Training deep neural networks directly on limited data increases the risk of **overfitting**, where the model memorizes training samples instead of learning generalized visual representations.

The augmentation operations used in this research are summarized in **Table 3.3**.

**Table 3.3 Data Augmentation Configuration**

| Augmentation Technique | Parameter      | Purpose                                         |
|------------------------|----------------|-------------------------------------------------|
| Rotation               | $\pm 30^\circ$ | Simulates different camera orientations         |
| Width Shift            | 20%            | Handles horizontal displacement                 |
| Height Shift           | 20%            | Handles vertical displacement                   |
| Shear Transformation   | 20%            | Introduces geometric variation                  |
| Zoom                   | 20%            | Simulates different camera distances            |
| Horizontal Flip        | Enabled        | Handles mirrored fruit orientation              |
| Fill Mode              | Nearest        | Preserves image continuity after transformation |

## 5.8 Training and Validation Data Generation

Following preprocessing and augmentation, TensorFlow's **ImageDataGenerator** was used to automatically generate batches of images during training. The dataset was divided into:

- **80% Training Set**

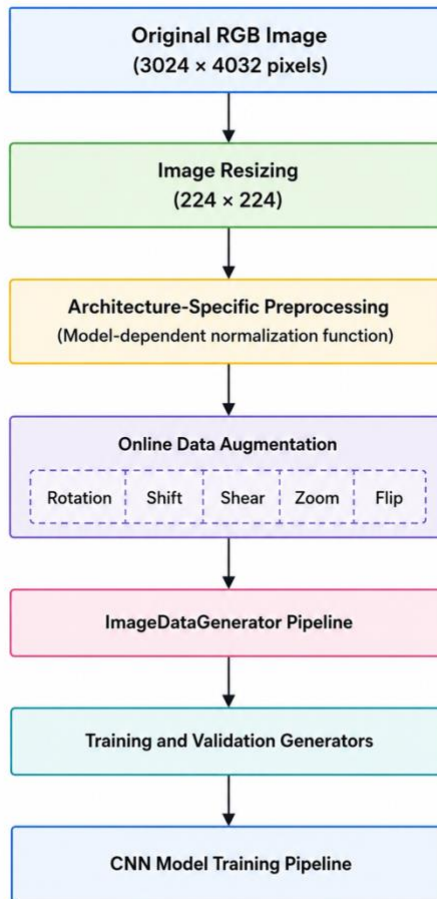


- **20% Validation Set**

using a fixed random seed to ensure reproducibility. The training generator applied both preprocessing and augmentation, whereas the validation generator performed only architecture-specific preprocessing without augmentation. This strategy ensures that validation performance accurately reflects the model's generalization capability on unseen data.

## 5.9 Workflow of the Preprocessing Pipeline

The complete preprocessing pipeline implemented in this research is illustrated conceptually in **Figure 5.9**.



**Figure 5.9:** Overview of the preprocessing and augmentation pipeline employed before CNN training.

## CHAPTER 6

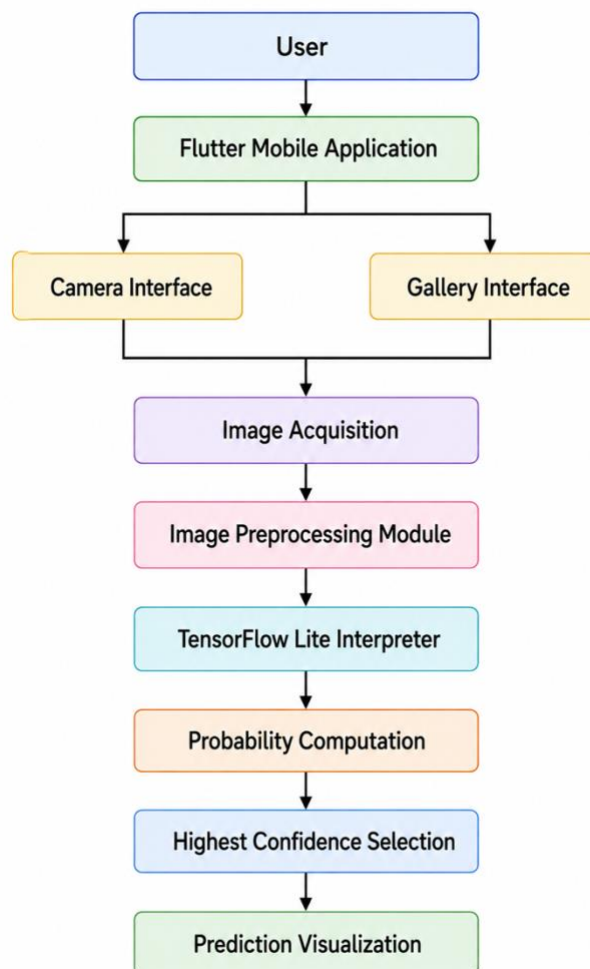
### MOBILE SYSTEM IMPLEMENTATION

Introduce the transition from the Trained Deep Learning model to the deployed Edge AI mobile application.

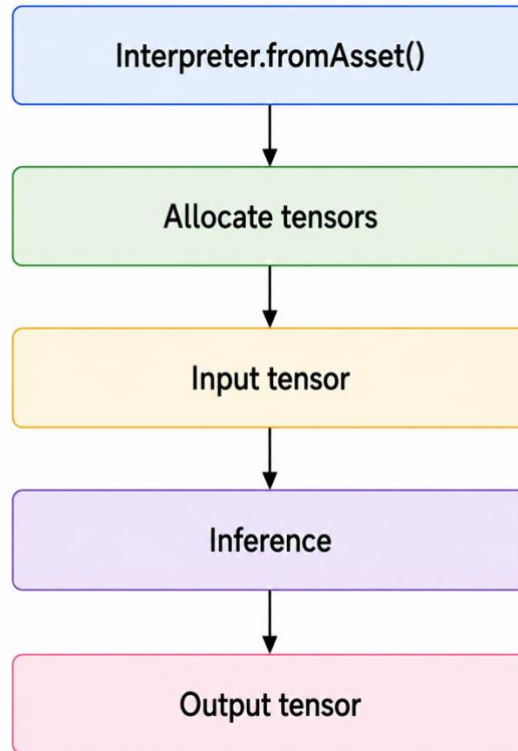
Explain that this chapter presents the complete implementation of the **AmmChini AI** mobile system, beginning with the Flutter application architecture, followed by the TensorFlow Lite inference pipeline, image preprocessing implementation, on-device prediction mechanism, and performance analysis. Emphasize that the objective is to transform the trained MobileNetV3-Small model into a practical, offline smartphone application capable of identifying indigenous mango varieties without relying on cloud computing or internet connectivity.

## 6.1 Cross-Platform Flutter Application Architecture

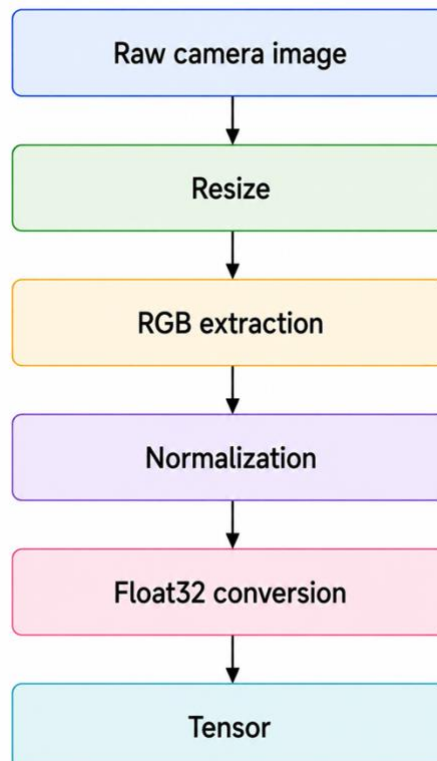
### 6.1.1 Overall System Architecture



### 6.1.2 TensorFlow Lite Interpreter



### 6.1.3 Need for Image Preprocessing



### 6.1.4 Mathematical Representation

Suppose

Original image

$$[I(x, y)]$$

Resize

$$[I_r(x, y)]$$

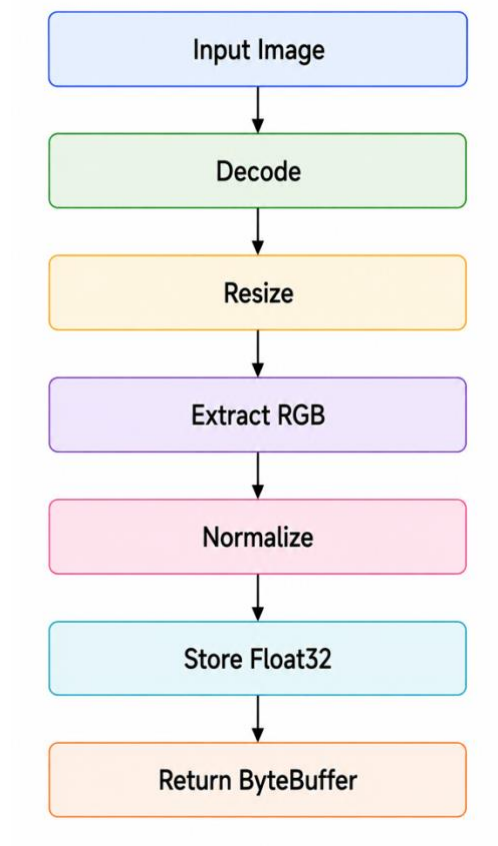
Normalize

$$[P = \frac{I}{255}]$$

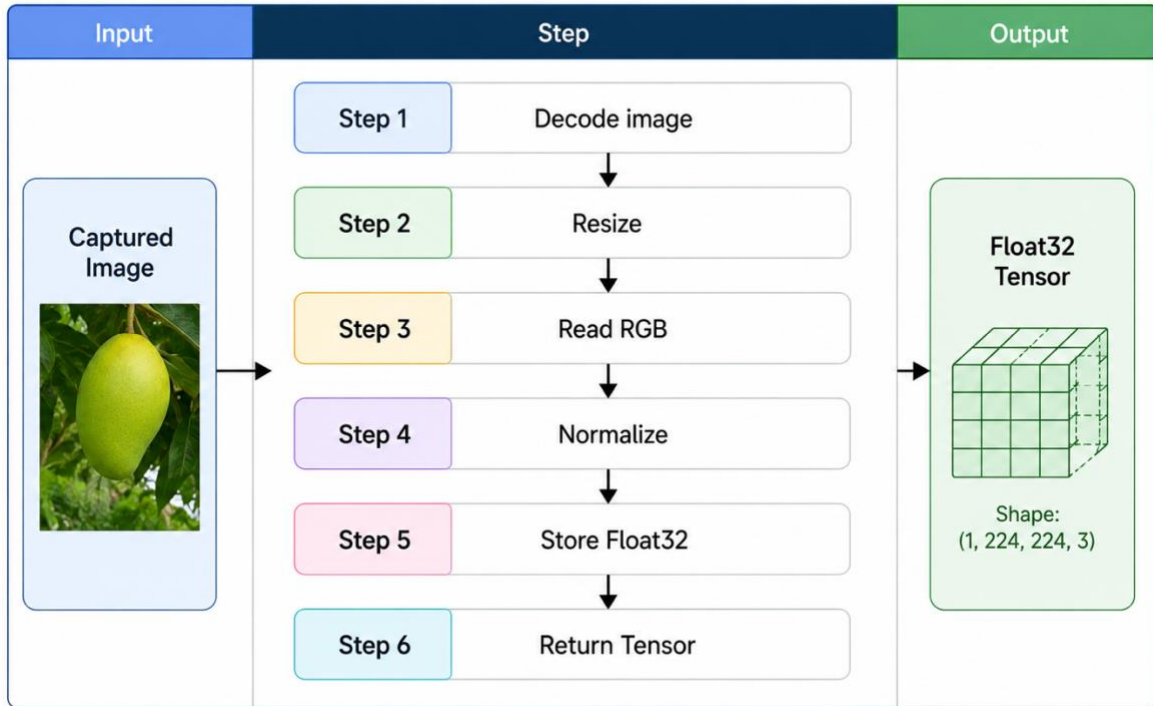
Tensor

$$[T \in R^{1 \times 224 \times 224 \times 3}]$$

### 6.1.5 ByteStream Conversion using \_imageToByteListFloat32()



## Image Preprocessing



### 6.1.6 PREDICTION RESULT VISUALIZATION

Display

- image
- predicted class
- confidence
- probability

User interaction.

### 6.1.7 INFERENCE LATENCY AND FIELD VIABILITY

This corresponds exactly to your outline.

Discuss

*AVERAGE INFERENCE TIME*

- Sub-second prediction
- Real-time performance

*OFFLINE EXECUTION*

- No Internet
- No Cloud
- No Server

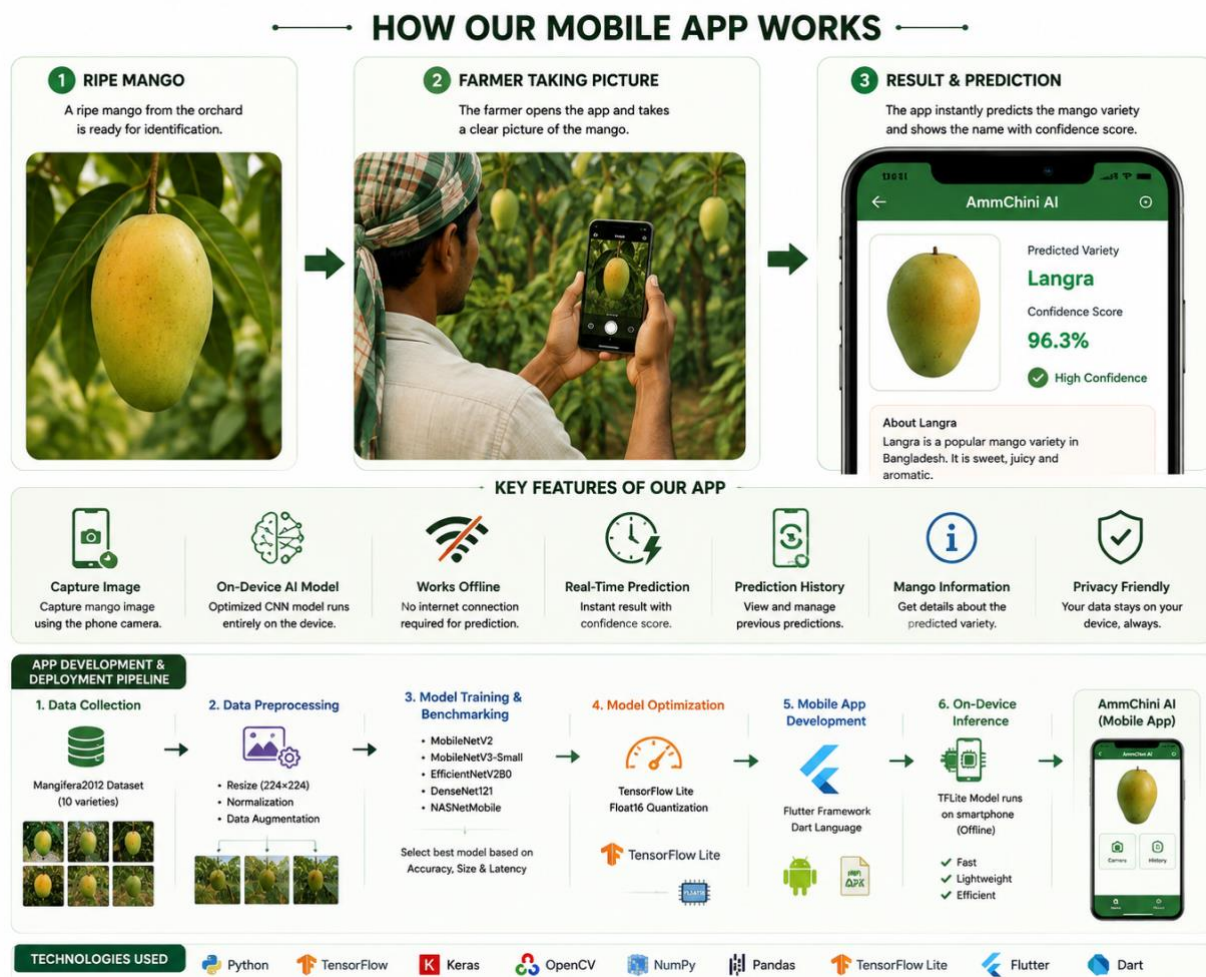
### ENERGY EFFICIENCY

- Low CPU usage
- Low RAM

### COST REDUCTION

- No API
- No Cloud Billing
- No Subscription

### RURAL DEPLOYMENT



## CHAPTER 7

### RESULT AND DISCUSSION

#### 7.1 Performance Benchmarking

The main goal of this research is to identify the appropriate lightweight convolutional neural network (CNN) for real-time offline mango variety classification on Android edge devices. While predictive accuracy remains an important performance indicator, deploying deep learning models on resource-constrained smartphones requires a broader evaluation that considers computational efficiency alongside classification performance. A model that achieves marginally higher accuracy may not necessarily be the most appropriate choice if it requires significantly larger storage, longer inference time, or greater computational resources. Therefore, this research adopts a comprehensive benchmarking methodology that evaluates multiple performance aspects before selecting the final deployment model.

To achieve an objective comparison, five transfer learning-based CNN architectures—**MobileNetV2**, **MobileNetV3-Small**, **EfficientNetV2B0**, **DenseNet121**, and **NASNetMobile**—were trained using the identical experimental configuration described in Chapter 3. Every architecture utilized the same **Mangifera2012 dataset**, identical preprocessing pipeline, online data augmentation strategy, training–validation split, optimizer configuration, batch size, learning rate, and early stopping criteria. Furthermore, all models employed the same custom classification head consisting of a **Global Average Pooling layer**, **Dropout (0.30)**, a **Dense layer with 128 neurons**, and a **Softmax output layer** for the ten mango varieties. Maintaining identical training conditions ensures that differences observed during evaluation originate from the architectural characteristics of the CNN models rather than inconsistencies in the experimental setup.

Unlike conventional image classification studies that rely solely on overall accuracy, this research performs a **deployment-oriented benchmarking analysis** by evaluating both predictive and computational performance. Four classification metrics were used to measure recognition capability:

- **Classification Accuracy**
- **Precision**

- **Recall**
- **Weighted F1-score**

Since the final objective is mobile deployment, additional engineering metrics were also considered during benchmarking:

- Model Storage Size (MB) – indicating the storage requirement of the trained model.
- Inference Latency (ms) – representing the average prediction time required for a single image.
- TensorFlow Lite Deployment Compatibility – evaluating the suitability of each architecture for Edge AI implementation.

By combining predictive performance with deployment efficiency, the benchmarking framework enables a balanced assessment of each CNN architecture

### Evaluation Metrics

For multi-class classification, overall accuracy is calculated as

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

where

- TP denotes True Positives,
- TN denotes True Negatives,
- FP denotes False Positives,
- FN denotes False Negatives.

Precision evaluates the reliability of positive predictions and is defined as

$$Precision = \frac{TP}{TP + FP}$$

Recall measures the model's ability to correctly identify all samples belonging to a particular class:

$$[Recall = \frac{TP}{TP + FN}]$$

The weighted F1-score combines precision and recall into a single performance measure while accounting for class imbalance:

$$F_1 = 2 \frac{Precision \times Recall}{Precision + Recall}$$



These metrics collectively provide a more reliable assessment than accuracy alone, particularly for fine-grained image classification tasks where visually similar classes may lead to different types of classification errors.

## Benchmarking Results

Table 4.1 summarizes the overall benchmarking results obtained from the five evaluated CNN architectures.

**Table 7.1 Comparative Performance of Evaluated CNN Architectures**

| Model                    | Accuracy (%)  | Precision (%) | Recall (%)   | Weighted F1 (%) | Model Size (MB) | Inference Latency (ms) |
|--------------------------|---------------|---------------|--------------|-----------------|-----------------|------------------------|
| MobileNetV2              | <b>80.25</b>  | <b>80.92</b>  | <b>80.25</b> | <b>80.33</b>    | <b>11.07</b>    | 34.66                  |
| <b>MobileNetV3-Small</b> | <b>88.50</b>  | <b>88.60</b>  | <b>88.50</b> | <b>88.27</b>    | <b>4.99</b>     | 28.29                  |
| EfficientNetV2B0         | <b>92.00</b>  | <b>92.08</b>  | <b>92.00</b> | <b>92.00</b>    | <b>24.39</b>    | 40.26                  |
| NASNetMobile             | <b>81.75</b>  | <b>81.78</b>  | <b>81.75</b> | <b>81.28</b>    | <b>20.48</b>    | 41.37                  |
| DenseNet121              | <b>90.259</b> | <b>90.54</b>  | <b>90.25</b> | <b>90.18</b>    | <b>29.82</b>    | 68.46                  |

## Comparative Analysis

### 7.2 Comparative Analysis

The experimental results indicate that **EfficientNetV2B0** achieved the best classification performance, with **92.00% accuracy** and the highest weighted F1-score. Its advanced feature extraction capability enabled effective discrimination of visually similar mango varieties. However, the larger model size and longer inference time limit its suitability for real-time deployment on resource-constrained mobile devices.

Among the lightweight models, **MobileNetV3-Small** achieved **88.5% accuracy**, only **3.5%** lower than EfficientNetV2B0. Despite this small performance gap, it offered the smallest model size and fastest inference speed, making it the most practical choice for Edge AI applications where storage, memory, and response time are critical.

**DenseNet121** achieved **90.25% accuracy**, demonstrating strong feature representation through dense connectivity. Nevertheless, its high storage requirement and slower inference reduce its effectiveness for smartphone-based deployment.

The baseline **MobileNetV2** obtained **80.25% accuracy** while maintaining computational efficiency through depthwise separable convolutions. However, the absence of architectural improvements such as squeeze-and-excitation modules and Hard-Swish activation resulted in lower recognition performance than MobileNetV3-Small.

**NASNetMobile** produced the lowest accuracy (**81.75%**) among the evaluated networks. Although generated through Neural Architecture Search, it failed to surpass MobileNetV3-Small and also required greater storage and inference time, making it less suitable for efficient edge deployment. Overall, the comparison highlights that **MobileNetV3-Small** provides the most balanced trade-off between classification accuracy, computational efficiency, model size, and inference latency. These characteristics make it the most appropriate architecture for offline, real-time mango variety classification on Android-based edge devices.

---

### 7.3 Deployment-Oriented Model Selection

Selecting an Edge AI model requires balancing classification performance with deployment efficiency rather than focusing solely on accuracy. Although **EfficientNetV2B0** achieved the highest accuracy, its larger model size, higher computational cost, and slower inference limit its practicality for offline mobile applications.

In contrast, **MobileNetV3-Small** delivered comparable recognition performance while requiring significantly fewer computational resources. Its compact architecture, low storage requirement, fast inference, and native compatibility with **TensorFlow Lite** make it well suited for Android-based edge devices used in agricultural environments.

Based on the overall benchmarking results, **MobileNetV3-Small** was selected as the deployment model for the proposed **AmmChini AI** system. The model was further optimized using **TensorFlow Lite Float16 post-training quantization** and integrated into the Flutter application to enable efficient, real-time offline mango variety classification.

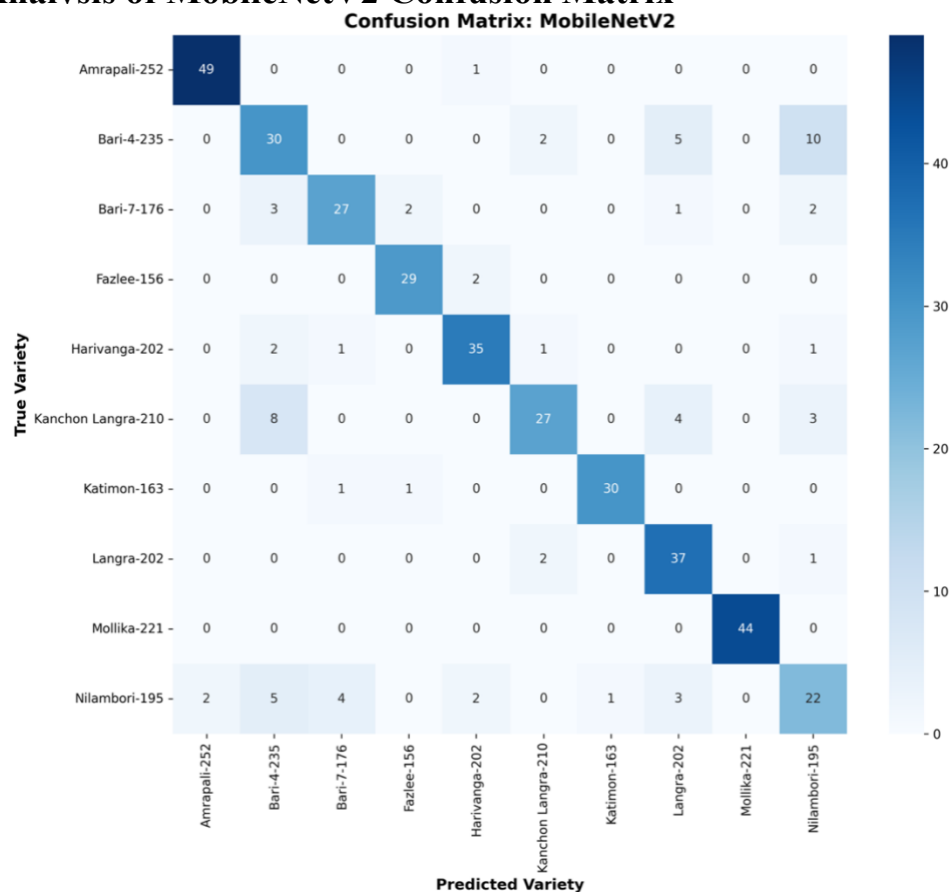
### Figure 7.2 Comparative Analysis of Class-wise F1-Scores Across Lightweight CNN Architectures



illustrates the comparative F1-score achieved by the five evaluated lightweight deep learning models: **MobileNetV2**, **MobileNetV3-Small**, **EfficientNetV2-B0**, **NASNetMobile**, and **DenseNet121**.

## 7.4 Analysis Confusion Matrix and Classification report for each model

### 7.4.1 Analysis of MobileNetV2 Confusion Matrix



## Key Observations

- **Mollika-221** achieved perfect classification (44/44).
- **Amrapali-252** correctly classified 49/50 samples.
- **Harivanga-202**, **Langra-202**, and **Katimon-163** also demonstrated strong recognition performance.
- **Bari-4-235**, **Kanchon Langra-210**, and **Nilambori-195** showed the highest misclassification rates.
- The most frequent confusion occurred between **Bari-4-235** ↔ **Nilambori-195** and **Kanchon Langra-210** ↔ **Bari-4-235**.
- The results indicate that **MobileNetV2** learned effective visual representations but struggled with highly similar mango varieties, explaining its lower accuracy than **MobileNetV3-Small**.

## 7.4.2 Classification Report Heatmap for MobileNetV2

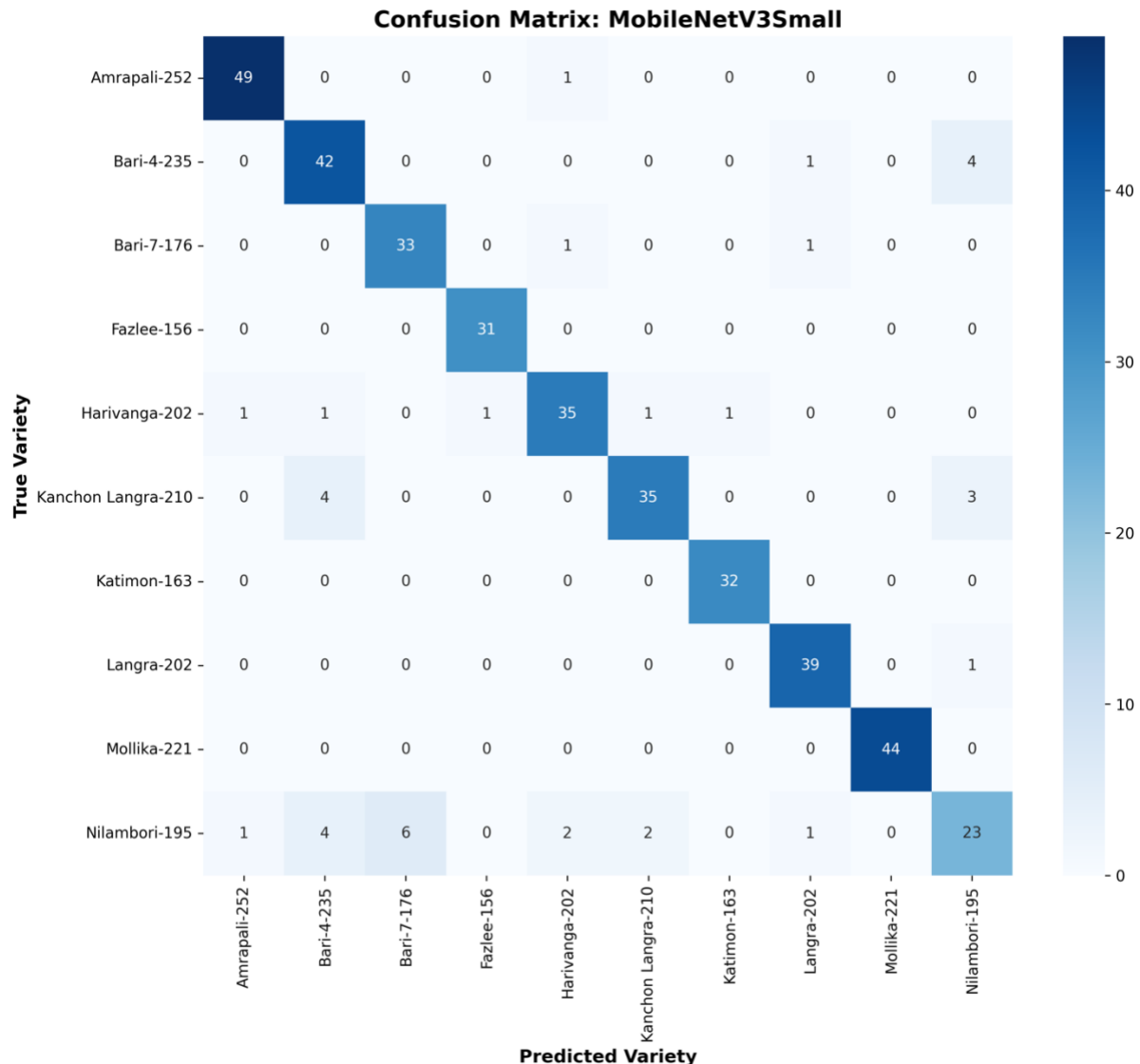
|                    | Classification Metrics: MobileNetV2 |        |          | Support |
|--------------------|-------------------------------------|--------|----------|---------|
|                    | Precision                           | Recall | F1-Score | Count   |
| Amrapali-252       | 0.96                                | 0.98   | 0.97     | 50      |
| Bari-4-235         | 0.62                                | 0.64   | 0.63     | 47      |
| Bari-7-176         | 0.82                                | 0.77   | 0.79     | 35      |
| Fazlee-156         | 0.91                                | 0.94   | 0.92     | 31      |
| Harivanga-202      | 0.88                                | 0.88   | 0.88     | 40      |
| Kanchon Langra-210 | 0.84                                | 0.64   | 0.73     | 42      |
| Katimon-163        | 0.97                                | 0.94   | 0.95     | 32      |
| Langra-202         | 0.74                                | 0.93   | 0.82     | 40      |
| Mollika-221        | 1.00                                | 1.00   | 1.00     | 44      |
| Nilambori-195      | 0.56                                | 0.56   | 0.56     | 39      |

## Key Observations

- **Mollika-221** achieved perfect classification performance with **Precision = 1.00**, **Recall = 1.00**, and **F1-score = 1.00**.
- **Amrapali-252**, **Katimon-163**, and **Fazlee-156** demonstrated consistently high classification metrics, indicating excellent feature separability.

- **Langra-202** achieved high recall (**0.93**) but comparatively lower precision (**0.74**), suggesting occasional false positive predictions.
- **Bari-4-235** and **Kanchon Langra-210** exhibited moderate performance due to frequent confusion with visually similar mango varieties.
- **Nilambori-195** recorded the lowest precision, recall, and F1-score (**0.56**), making it the most challenging class for MobileNetV2.
- The class-wise evaluation reinforces the findings from the confusion matrix and demonstrates that MobileNetV2, while computationally efficient, has limited capability in distinguishing highly similar indigenous Bangladeshi mango varieties. These limitations justify the subsequent investigation of more advanced lightweight architectures for Edge AI deployment.

### 7.4.3 Confusion Matrix for the MobileNetV3-Small Model



## Key Observations

- **Fazlee-156 (31/31), Katimon-163 (32/32), and Mollika-221 (44/44)** achieved **perfect classification**, with no observed misclassifications.
- **Amrapali-252** maintained excellent performance with **49 correct predictions out of 50**.
- **Langra-202** correctly classified **39 of 40** validation samples, demonstrating highly reliable recognition.
- **Bari-4-235** and **Kanchon Langra-210** exhibited substantial improvement compared with MobileNetV2, indicating enhanced feature discrimination.
- **Nilambori-195** remained the most challenging variety; however, the number of correct predictions increased relative to the MobileNetV2 model.
- The most frequent remaining confusion occurred between **Bari-4-235** and **Nilambori-195**, as well as **Kanchon Langra-210** and **Bari-4-235**, reflecting their close morphological similarity.
- Overall, the confusion matrix demonstrates that **MobileNetV3-Small significantly reduces inter-class confusion while maintaining computational efficiency**, providing strong evidence for its selection as the final model for TensorFlow Lite optimization and deployment in the proposed **AmmChini AI Edge AI system**.

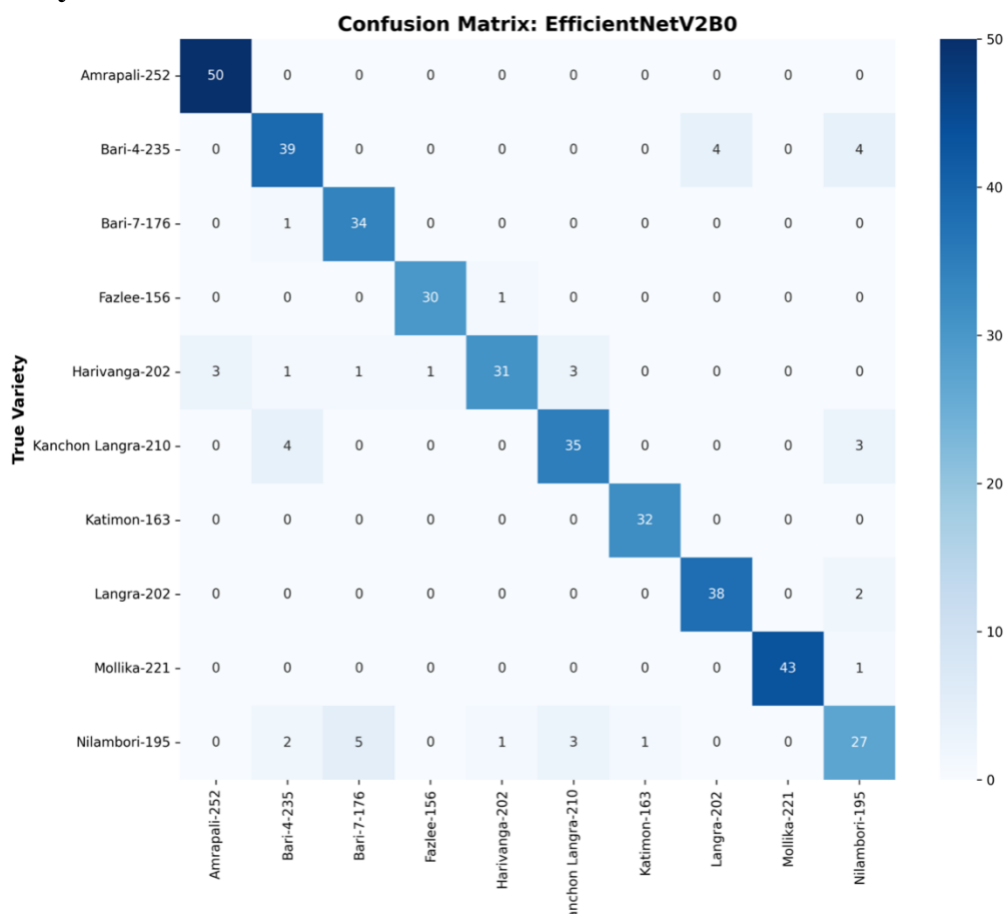
## 7.4.4 Classification Report Heatmap for the Proposed MobileNetV3-Small Model

| Classification Metrics: MobileNetV3Small |           |        |          | Support |
|------------------------------------------|-----------|--------|----------|---------|
| Amrapali-252                             | 0.96      | 0.98   | 0.97     | 50      |
| Bari-4-235                               | 0.82      | 0.89   | 0.86     | 47      |
| Bari-7-176                               | 0.85      | 0.94   | 0.89     | 35      |
| Fazlee-156                               | 0.97      | 1.00   | 0.98     | 31      |
| Harivanga-202                            | 0.90      | 0.88   | 0.89     | 40      |
| Kanchon Langra-210                       | 0.92      | 0.83   | 0.88     | 42      |
| Katimon-163                              | 0.97      | 1.00   | 0.98     | 32      |
| Langra-202                               | 0.93      | 0.97   | 0.95     | 40      |
| Mollika-221                              | 1.00      | 1.00   | 1.00     | 44      |
| Nilambori-195                            | 0.74      | 0.59   | 0.66     | 39      |
|                                          | Precision | Recall | F1-Score | Count   |

Key Observations

- **Mollika-221** achieved **perfect classification** with **Precision = 1.00**, **Recall = 1.00**, and **F1-score = 1.00**.
- **Fazlee-156** and **Katimon-163** achieved excellent performance with **F1-scores of 0.98**, reflecting highly reliable recognition.
- **Amrapali-252** and **Langra-202** maintained outstanding classification performance, both achieving **F1-scores above 0.95**.
- **Bari-4-235** improved significantly from an **F1-score of 0.63 (MobileNetV2)** to **0.86**, demonstrating enhanced feature discrimination.
- **Kanchon Langra-210** also exhibited a notable improvement, increasing its **F1-score from 0.73 to 0.88**.
- **Nilambori-195** remained the most difficult class; however, its classification performance improved compared with the baseline MobileNetV2 model.
- Overall, the classification report heatmap confirms that the proposed **MobileNetV3-Small** architecture delivers **more balanced and robust class-wise performance** than the baseline MobileNetV2 while maintaining the lightweight characteristics required for efficient **Edge AI deployment**. These findings strongly support its selection as the final model integrated into the AmmChini AI mobile application.

7.4.5 Analysis of the EfficientNetV2-B0 Confusion Matrix



## Key Observations

- **Amrapali-252** and **Katimon-163** achieved **perfect classification**, with all validation samples correctly recognized.
- **Bari-7-176** achieved **34 correct predictions out of 35**, demonstrating excellent recognition capability.
- **Langra-202** correctly classified **38 of 40** validation images, indicating strong class separability.
- **Mollika-221** achieved near-perfect performance with **43 correct predictions out of 44**.
- **Nilambori-195** remained the most challenging class; however, EfficientNetV2-B0 achieved the **highest recognition rate for this variety** among all evaluated models.
- The most frequent remaining confusion occurred between **Bari-4-235**, **Langra-202**, and **Nilambori-195**, reflecting their close morphological similarity.
- Although EfficientNetV2-B0 achieved the **highest overall classification accuracy (92.00%)**, its larger model size and longer inference time reduce its practicality for resource-constrained mobile devices. These observations justify the subsequent selection of **MobileNetV3-Small** as the final deployment model for the proposed **AmmChini AI Edge AI** application.

## 7.4.6 Classification Report Heatmap for EfficientNetV2-B0

| Classification Metrics: EfficientNetV2B0 |           |        |          | Support |
|------------------------------------------|-----------|--------|----------|---------|
| Amrapali-252                             | 0.94      | 1.00   | 0.97     | 50      |
| Bari-4-235                               | 0.83      | 0.83   | 0.83     | 47      |
| Bari-7-176                               | 0.85      | 0.97   | 0.91     | 35      |
| Fazlee-156                               | 0.97      | 0.97   | 0.97     | 31      |
| Harivanga-202                            | 0.94      | 0.78   | 0.85     | 40      |
| Kanchon Langra-210                       | 0.85      | 0.83   | 0.84     | 42      |
| Katimon-163                              | 0.97      | 1.00   | 0.98     | 32      |
| Langra-202                               | 0.90      | 0.95   | 0.93     | 40      |
| Mollika-221                              | 1.00      | 0.98   | 0.99     | 44      |
| Nilambori-195                            | 0.73      | 0.69   | 0.71     | 39      |
|                                          | Precision | Recall | F1-Score | Count   |

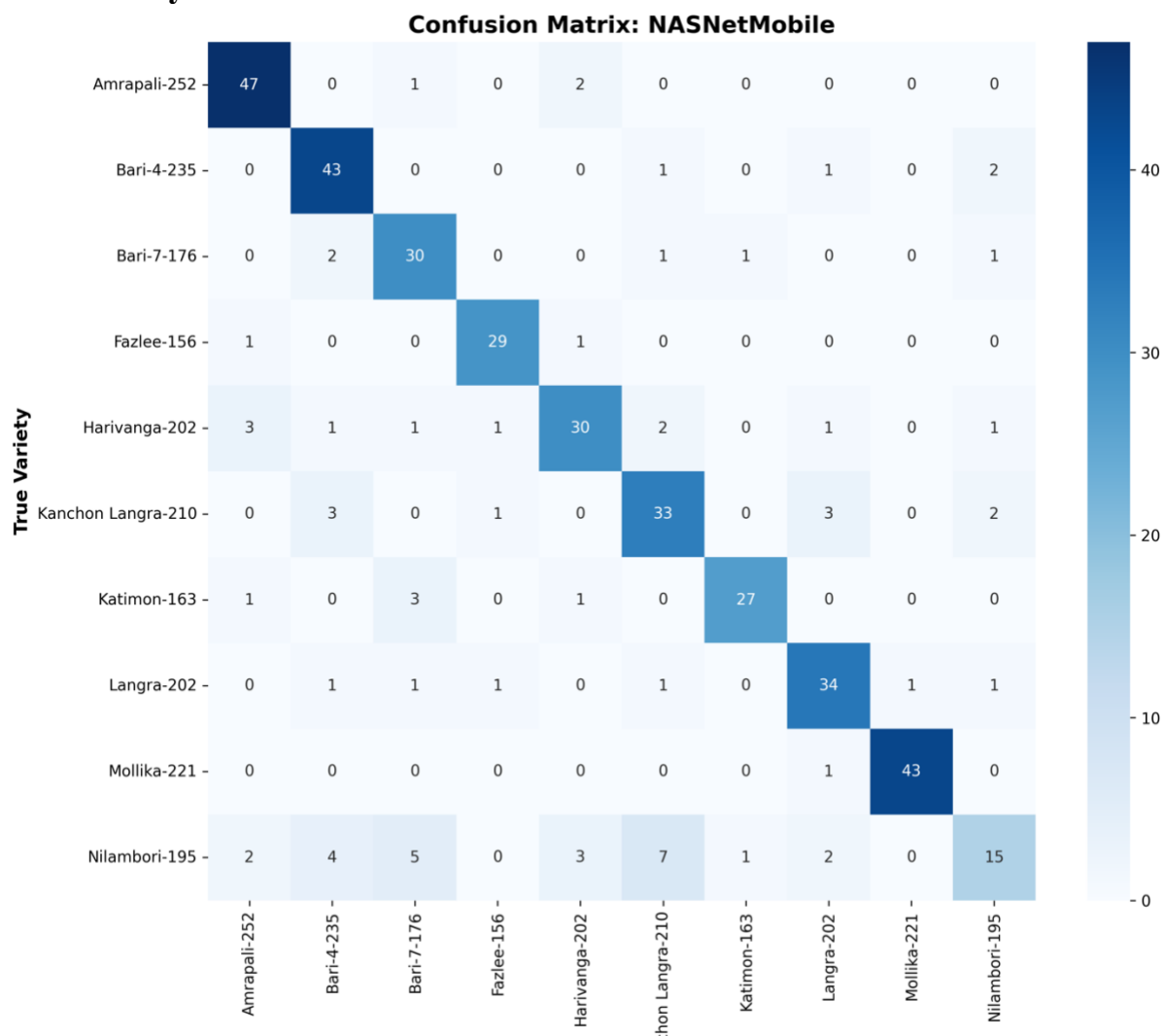
## Key Observations

- **Katimon-163** achieved **Recall = 1.00** and an **F1-score of 0.98**, indicating perfect identification of all validation samples.



- **Amrapali-252** achieved **Recall = 1.00** with an **F1-score of 0.97**, demonstrating excellent recognition capability.
- **Mollika-221** obtained **Precision = 1.00** and an **F1-score of 0.99**, reflecting near-perfect classification.
- **Fazlee-156** maintained balanced performance with **Precision = Recall = 0.97**.
- **Langra-202** and **Bari-7-176** both achieved **F1-scores above 0.90**, demonstrating strong class discrimination.
- **Nilambori-195** remained the most challenging class but still achieved its **highest F1-score (0.71)** among all evaluated architectures.
- EfficientNetV2-B0 delivered the **highest weighted Precision (0.9208)**, **Recall (0.9200)**, and **F1-score (0.9200)** across all benchmark models, confirming its superior predictive capability.
- Despite its outstanding classification performance, the model's **larger memory footprint and higher inference latency** limit its suitability for real-time Edge AI deployment, reinforcing the selection of **MobileNetV3-Small** as the final deployment model for the **AmmChini AI** mobile application.

## 7.4.7 Analysis of the NASNetMobile Confusion Matrix



## Key Observations

- **Amrapali-252** achieved excellent recognition performance, with **47 of 50** validation images correctly classified.
- **Bari-4-235** and **Mollika-221** both correctly classified **43 validation samples**, demonstrating relatively stable recognition performance.
- **Langra-202** achieved **34 correct predictions out of 40**, indicating moderate classification capability.
- **Harivanga-202**, **Katimon-163**, and **Kanchon Langra-210** experienced noticeably higher misclassification rates than those observed in MobileNetV3-Small and EfficientNetV2-B0.
- **Nilambori-195** remained the most difficult class, with only **15 correct predictions out of 39**, representing the lowest class-wise recognition performance among all benchmark models.
- The most frequent confusion occurred between **Nilambori-195**, **Bari-7-176**, **Harivanga-202**, and **Kanchon Langra-210**, reflecting their close morphological similarity.
- Although NASNetMobile was designed for mobile applications, its **81.75% classification accuracy**, **20.48 MB model size**, and **41.37 ms inference latency** provide a less favorable trade-off between accuracy and computational efficiency than the proposed **MobileNetV3-Small** architecture, making it less suitable for deployment in the **AmmChini AI** offline Edge AI application.

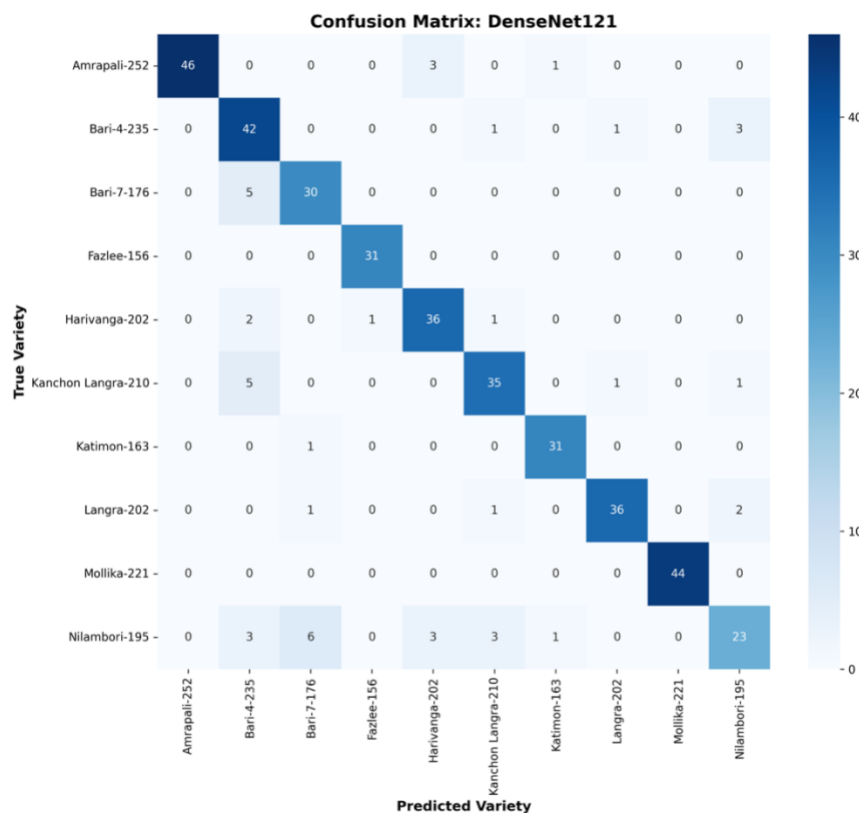
## 7.4.8 Classification Report Heatmap for NASNetMobile

|                    | Classification Metrics: NASNetMobile |        |          | Support |
|--------------------|--------------------------------------|--------|----------|---------|
|                    | Precision                            | Recall | F1-Score | Count   |
| Amrapali-252       | 0.87                                 | 0.94   | 0.90     | 50      |
| Bari-4-235         | 0.80                                 | 0.91   | 0.85     | 47      |
| Bari-7-176         | 0.73                                 | 0.86   | 0.79     | 35      |
| Fazlee-156         | 0.91                                 | 0.94   | 0.92     | 31      |
| Harivanga-202      | 0.81                                 | 0.75   | 0.78     | 40      |
| Kanchon Langra-210 | 0.73                                 | 0.79   | 0.76     | 42      |
| Katimon-163        | 0.93                                 | 0.84   | 0.89     | 32      |
| Langra-202         | 0.81                                 | 0.85   | 0.83     | 40      |
| Mollika-221        | 0.98                                 | 0.98   | 0.98     | 44      |
| Nilambori-195      | 0.68                                 | 0.38   | 0.49     | 39      |

## Key Observations

- **Mollika-221** achieved the best overall performance with **Precision = 0.98, Recall = 0.98, and F1-score = 0.98**, indicating highly reliable classification.
- **Fazlee-156, Amrapali-252, and Katimon-163** also demonstrated strong predictive performance, each achieving **F1-scores close to or above 0.89**.
- **Bari-4-235** and **Langra-202** exhibited balanced classification performance with F1-scores of **0.85** and **0.83**, respectively.
- **Harivanga-202, Kanchon Langra-210, and Bari-7-176** showed comparatively lower F1-scores, reflecting increased inter-class confusion.
- **Nilambori-195** recorded the lowest performance (**Precision = 0.68, Recall = 0.38, F1-score = 0.49**), confirming it as the most difficult mango variety to classify.
- Compared with **MobileNetV3-Small** and **EfficientNetV2-B0**, NASNetMobile exhibited greater variation in class-wise metrics, indicating weaker feature discrimination for fine-grained indigenous mango classification.
- The classification report further supports the conclusion that **NASNetMobile provides acceptable baseline performance but does not achieve the optimal balance of classification accuracy, robustness, and deployment efficiency required for the proposed AmmChini AI edge-based mobile application.**

## 7.4.9 Analysis of the DenseNet121 Confusion Matrix



## Key Observations

- **Fazlee-156** and **Mollika-221** achieved **100% classification accuracy**, with every validation sample correctly identified.
- **Harivanga-202**, **Langra-202**, **Katimon-163**, and **Bari-4-235** demonstrated consistently strong classification performance with only a few misclassified samples.
- **Amrapali-252** achieved high recognition accuracy, with most prediction errors occurring toward **Harivanga-202**.
- **Kanchon Langra-210** and **Bari-7-176** exhibited moderate confusion, primarily with visually similar neighboring varieties.
- **Nilambori-195** remained the most difficult class, with misclassifications distributed across multiple mango varieties, indicating substantial feature overlap.
- The confusion matrix confirms that DenseNet121 performs effectively for most indigenous mango varieties but still encounters challenges in distinguishing morphologically similar cultivars.
- Compared with the proposed **MobileNetV3-Small** model, DenseNet121 provides competitive recognition capability but exhibits greater inter-class confusion while requiring significantly higher computational resources for deployment on resource-constrained mobile devices.

## 7.4.10 Classification Report Heatmap for DenseNet121

| Classification Metrics: DenseNet121 |           |        |          | Support |
|-------------------------------------|-----------|--------|----------|---------|
| Amrapali-252                        | 1.00      | 0.92   | 0.96     | 50      |
| Bari-4-235                          | 0.74      | 0.89   | 0.81     | 47      |
| Bari-7-176                          | 0.79      | 0.86   | 0.82     | 35      |
| Fazlee-156                          | 0.97      | 1.00   | 0.98     | 31      |
| Harivanga-202                       | 0.86      | 0.90   | 0.88     | 40      |
| Kanchon Langra-210                  | 0.85      | 0.83   | 0.84     | 42      |
| Katimon-163                         | 0.94      | 0.97   | 0.95     | 32      |
| Langra-202                          | 0.95      | 0.90   | 0.92     | 40      |
| Mollika-221                         | 1.00      | 1.00   | 1.00     | 44      |
| Nilambori-195                       | 0.79      | 0.59   | 0.68     | 39      |
|                                     | Precision | Recall | F1-Score | Count   |

## Key Observations

- **Mollika-221** achieved perfect classification with **Precision = 1.00, Recall = 1.00, and F1-score = 1.00**, representing the best-performing class.
- **Fazlee-156, Amrapali-252, Katimon-163, and Langra-202** also demonstrated excellent classification performance, each achieving **F1-scores above 0.92**.
- **Harivanga-202** maintained balanced prediction capability with an **F1-score of 0.88**, indicating reliable feature discrimination.
- **Kanchon Langra-210, Bari-7-176, and Bari-4-235** exhibited moderate performance due to increased confusion with visually similar varieties.
- **Nilambori-195** remained the most challenging class, recording the lowest **Recall (0.59)** and **F1-score (0.68)** among all categories.
- DenseNet121 successfully learned discriminative representations for most indigenous mango varieties but exhibited reduced consistency for morphologically similar cultivars.
- Compared with the proposed **MobileNetV3-Small** model, DenseNet121 achieved competitive classification accuracy but offered lower deployment efficiency because of its significantly larger computational complexity, parameter count, and inference cost on mobile edge devices.

## **CHAPTER 8**

### **CONCLUSION AND FUTURE WORK**

#### **8.1 Research Summary**

The rapid advancement of artificial intelligence and mobile computing has created new opportunities for developing intelligent agricultural decision-support systems capable of operating directly on edge devices. Nevertheless, many existing computer vision applications for crop identification continue to rely on computationally intensive deep learning architectures and cloud-based inference services, limiting their applicability in rural agricultural environments where internet connectivity, computational resources, and operational costs remain significant challenges. Motivated by these limitations, this research investigated the feasibility of developing an efficient, lightweight, and completely offline mango variety classification system specifically designed for real-world deployment on Android smartphones.

The main goal of this research was to design and developed an end-to-end Edge AI framework capable of accurately recognizing indigenous Bangladeshi mango varieties while maintaining low computational complexity suitable for mobile devices. To accomplish this objective, a comprehensive dataset consisting of ten widely cultivated indigenous mango varieties was collected and carefully organized. Prior to model training, all images underwent standardized preprocessing and data augmentation techniques to improve dataset diversity and reduce the likelihood of model overfitting. Dynamic preprocessing pipelines were also developed to ensure compatibility with the varying input requirements of different convolutional neural network architectures.

Following model selection, the optimized MobileNetV3-Small network was converted into TensorFlow Lite format using Float16 post-training quantization. This optimization substantially reduced model size while preserving classification performance, making the model appropriate for real-time smartphone inference. The quantized model was subsequently integrated into a Flutter-based cross-platform mobile application named AmmChini AI. The application was designed to perform the complete inference pipeline—including image acquisition, preprocessing, TensorFlow Lite inference, confidence estimation, and result visualization—entirely on the local device without requiring any external server, cloud service, or internet connection

## 8.2 Future Work

Although the proposed AmmChini AI system demonstrates promising performance for offline mango variety classification, several opportunities remain for extending the research and improving the overall capabilities of the system. Future developments can focus on expanding the dataset, improving model robustness, incorporating additional computer vision tasks, and integrating advanced Edge AI optimization techniques to support broader agricultural applications

Another promising extension involves incorporating object detection and instance segmentation techniques. Rather than requiring users to manually center a mango before prediction, future systems could automatically detect multiple mangoes within a single image and classify each fruit independently. Such functionality would enable batch analysis during harvesting, packaging, grading, and quality inspection processes, thereby increasing operational efficiency.

Future research may also investigate disease diagnosis and fruit quality assessment within the same application. By combining variety classification with disease detection, ripeness estimation, pest identification, and external defect analysis, the application could evolve into a comprehensive intelligent decision-support platform for mango production. Multi-task learning architectures may allow these related tasks to share common feature representations while maintaining computational efficiency

Although Float16 quantization achieved an effective balance between model size and prediction accuracy, additional optimization techniques such as Integer (INT8) quantization, quantization-aware training, structured pruning, neural architecture search, and knowledge distillation may further reduce computational requirements without substantial performance degradation. These

# References

- [1] O. Salazar-Campose et al., “Deep learning approach for automated ‘Kent’ mango maturity classification,” *Result in Control and Optimization*, vol. 20, 2025 .
- [2] Y. Wu, H. Hsu and S. Chen, “Mobile-Based Deep Learning Approach for Multi-Object Aiwon Mnago Grade Classification,” *Electronics*, vol. 14, 2025
- [3] A. Howard *et al.*, "Searching for MobileNetV3," *Proc. IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 1314–1324, 2019.
- [4] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," *Proc. ICML*, 2019.
- [5] M. Tan and Q. V. Le, "EfficientNetV2: Smaller Models and Faster Training," *Proc. ICML*, 2021.
- [6] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely Connected Convolutional Networks," *Proc. CVPR*, 2017.
- [7] B. Zoph, V. Vasudevan, J. Shlens, and Q. V. Le, "Learning Transferable Architectures for Scalable Image Recognition," *Proc. CVPR*, 2018.
- [8] L. Chuquimarca et al., “Classifying Healthy and Defective Fruits with a Multi-Input Architecture,” *arXiv preprint*, 2024
- [9] H. Peng et al., “Attention-based ResNet for Multi-class Fruit Detection,” *Computers and Electronics in Agriculture*, vol. 205, 2023
- [10] H. Peng et al., “Attention-based ResNet for Multi-class Fruit Detection,” *Computers and Electronics in Agriculture*, vol. 205, 2023.
- [11] T. Nguyen and S. Lee, “EfficientNet vs VGG16: A benchmark for mango leaf disease detection,” *IEEE Access*, vol. 10, 2022.
- [12] M. Islam et al., “Mango Variety Classification Using a Shallow CNN Architecture,” *International Journal of Advanced Computer Science and Applications*, vol. 12, no. 8, 2021.



- [13] S. Kumar and V. Singh, "Automated Grading of Alphonso Mangoes using InceptionV3," *Journal of Food Engineering*, vol. 310, 2023.
- [14] L. Zhang et al., "Real-time Apple Defect Detection using GhostNet on Embedded Systems," *Sensors*, vol. 24, 2024.
- [15] K. Ahmed et al., "Mango Fruit Counting in Orchards using Mask R-CNN," *Precision Agriculture*, vol. 23, 2022.
- [17] P. Das and S. Roy, "Transfer Learning Approaches for Mango Rot Detection," *Ecological Informatics*, vol. 64, 2021.
- [18] N. Patel et al., "Smartphone-based Ripeness Estimation of Mangoes," *Computers and Electronics in Agriculture*, vol. 175, 2020.
- [19] Q. Zhao et al., "Model Pruning of VGG16 for Robotic Fruit Sorting," *IEEE Transactions on Industrial Electronics*, vol. 70, 2023.
- [20] Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [21] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media, 2019.
- [22] A. N. Bhatti et al., "A Lightweight Mobile Deep Learning Framework for Real-Time Plant Disease Detection in Smart Agriculture," *2025 IEEE International Symposium on Innovative Approaches in Smart Technologies (ISAS)*, 2025.
- [23] T. Njoroge, R. Kibuku, and K. Mugoye, "Comparative and Edge-Hybrid Modeling of EfficientNetV2 and MobileNetV2 for Multi-Class Crop Disease Classification," *Journal of Engineering Computing*, 2025.
- [24] S. Rahayu and S. F. Romdoni, "Bayesian Optimized Pretrained CNNs for Mango Leaf Disease Classification: A Comparative Study," *Jurnal Teknik Informatika*, 2025.
- [25] S. dos Santos et al., "A Comparative Analysis of Lightweight CNNs for Robust and Efficient Mango Leaf Disease Classification," *SSRN*, 2026.
- [26] N. Paul, "Deep Learning Application in Plant Stress Detection," Ph.D. dissertation, North Dakota State University.